

Deep Reinforcement Learning with Double Q-learning

Hado van Hasselt and Arthur Guez and David Silver

Google DeepMind

Abstract

The popular Q-learning algorithm is known to overestimate action values under certain conditions. It was not previously known whether, in practice, such overestimations are common, whether they harm performance, and whether they can generally be prevented. In this paper, we answer all these questions affirmatively. In particular, we first show that the recent DQN algorithm, which combines Q-learning with a deep neural network, suffers from substantial overestimations in some games in the Atari 2600 domain. We then show that the idea behind the Double Q-learning algorithm, which was introduced in a tabular setting, can be generalized to work with large-scale function approximation. We propose a specific adaptation to the DQN algorithm and show that the resulting algorithm not only reduces the observed overestimations, as hypothesized, but that this also leads to much better performance on several games.

The goal of reinforcement learning (Sutton and Barto, 1998) is to learn good policies for sequential decision problems, by optimizing a cumulative future reward signal. Q-learning (Watkins, 1989) is one of the most popular reinforcement learning algorithms, but it is known to sometimes learn unrealistically high action values because it includes a maximization step over estimated action values, which tends to prefer overestimated to underestimated values.

In previous work, overestimations have been attributed to insufficiently flexible function approximation (Thrun and Schwartz, 1993) and noise (van Hasselt, 2010, 2011). In this paper, we unify these views and show overestimations can occur when the action values are inaccurate, irrespective of the source of approximation error. Of course, imprecise value estimates are the norm during learning, which indicates that overestimations may be much more common than previously appreciated.

It is an open question whether, if the overestimations do occur, this negatively affects performance in practice. Overoptimistic value estimates are not necessarily a problem in and of themselves. If all values would be uniformly higher then the relative action preferences are preserved and we would not expect the resulting policy to be any worse. Furthermore, it is known that sometimes it is good to be optimistic: optimism in the face of uncertainty is a well-known

exploration technique (Kaelbling et al., 1996). If, however, the overestimations are not uniform and not concentrated at states about which we wish to learn more, then they might negatively affect the quality of the resulting policy. Thrun and Schwartz (1993) give specific examples in which this leads to suboptimal policies, even asymptotically.

To test whether overestimations occur in practice and at scale, we investigate the performance of the recent DQN algorithm (Mnih et al., 2015). DQN combines Q-learning with a flexible deep neural network and was tested on a varied and large set of deterministic Atari 2600 games, reaching human-level performance on many games. In some ways, this setting is a best-case scenario for Q-learning, because the deep neural network provides flexible function approximation with the potential for a low asymptotic approximation error, and the determinism of the environments prevents the harmful effects of noise. Perhaps surprisingly, we show that even in this comparatively favorable setting DQN sometimes substantially overestimates the values of the actions.

We show that the idea behind the Double Q-learning algorithm (van Hasselt, 2010), which was first proposed in a tabular setting, can be generalized to work with arbitrary function approximation, including deep neural networks. We use this to construct a new algorithm we call Double DQN. We then show that this algorithm not only yields more accurate value estimates, but leads to much higher scores on several games. This demonstrates that the overestimations of DQN were indeed leading to poorer policies and that it is beneficial to reduce them. In addition, by improving upon DQN we obtain state-of-the-art results on the Atari domain.

Background

To solve sequential decision problems we can learn estimates for the optimal value of each action, defined as the expected sum of future rewards when taking that action and following the optimal policy thereafter. Under a given policy π , the true value of an action a in a state s is

$$Q_{\pi}(s, a) \equiv \mathbb{E}[R_1 + \gamma R_2 + \dots \mid S_0 = s, A_0 = a, \pi],$$

where $\gamma \in [0, 1]$ is a discount factor that trades off the importance of immediate and later rewards. The optimal value is then $Q_*(s, a) = \max_{\pi} Q_{\pi}(s, a)$. An optimal policy is easily derived from the optimal values by selecting the highest-valued action in each state.

使用双 Q 学习的深度强化学习

Hado van Hasselt、Arthur Guez 和 David Silver Google
DeepMind

抽象的

众所周知，流行的 Q 学习算法会在某些条件下高估动作值。此前尚不清楚这种高估在实践中是否常见，是否会损害绩效，以及通常是否可以预防。在本文中，我们对所有这些问题做出了肯定的回答。特别是，我们首先表明，最近的 DQN 算法将 Q 学习与深度神经网络相结合，但在 Atari 2600 领域的某些游戏中存在严重高估的问题。然后我们展示了在表格设置中引入的双 Q 学习算法背后的思想可以推广到大规模函数逼近。我们提出了对 DQN 算法的具体调整，并表明所得到的算法不仅减少了对所观察到的高估（如假设的那样），而且这还导致在一些游戏中获得更好的性能。

强化学习（Sutton 和 Barto，1998）的目标是通过优化累积的未来奖励信号来学习顺序决策问题的良好策略。Q-learning（Watkins，1989）是最流行的强化学习算法之一，但众所周知，它有时会学习到不切实际的高动作值，因为它包含对估计动作值的最大化步骤，这往往更喜欢高估而不是低估的值。

在之前的工作中，高估被归因于不够灵活的函数逼近（Thrun 和 Schwartz，1993）和噪声（van Hasselt，2010，2011）。在本文中，我们统一了这些观点，并表明当动作值不准确时，无论近似误差的来源如何，都可能发生高估。当然，不精确的价值估计是学习过程中的常态，这表明高估可能比以前意识到的更为常见。

如果确实发生高估，是否会对实践中的表现产生负面影响，这是一个悬而未决的问题。过度乐观的价值估计本身并不一定是一个问题。如果所有值都一致较高，那么相对行动偏好就会被保留，我们不会期望最终的政策会变得更糟。此外，众所周知，有时保持乐观是有好处的：面对不确定性保持乐观是众所周知的。

勘探技术（Kaelbling 等，1996）。然而，如果高估不统一并且不集中在我们希望了解更多信息的州，那么它们可能会对最终政策的质量产生负面影响。Thrun 和 Schwartz（1993）给出了具体的例子，其中这会导致次优的政策，甚至是渐近的。

为了测试在实践中和规模上是否发生高估，我们研究了最近的 DQN 算法的性能（Mnih 等人，2015）。DQN 将 Q 学习与灵活的深度神经网络相结合，并在大量确定性 Atari 2600 游戏上进行了测试，在许多游戏中达到了人类水平的表现。在某些方面，这种设置是 Q 学习的最佳情况，因为深度神经网络提供了灵活的函数逼近，具有低渐近逼近误差的潜力，并且环境的确定性可以防止噪声的有害影响。也许令人惊讶的是，我们发现即使在这种相对有利的环境中，DQN 有时也会大大高估动作的价值。

我们证明了双 Q 学习算法（van Hasselt，2010）背后的思想（该算法首次在表格设置中提出）可以推广到任意函数逼近，包括深度神经网络。我们用它来构建一个称为 Double DQN 的新算法。然后我们证明该算法不仅可以产生更准确的价值估计，而且可以在多个游戏中获得更高的分数。这表明对 DQN 的高估确实导致了更糟糕的政策，并且减少它们是有益的。此外，通过改进 DQN，我们在 Atari 领域获得了最先进的结果。

背景

为了解决顺序决策问题，我们可以学习对每个行动的最优价值的估计，定义为采取该行动并随后遵循最优策略时未来奖励的预期总和。在给定的策略 π 下，状态 s 下操作 a 的真实值为

$$Q_{\pi}(s, a) \equiv \mathbb{E}[R_1 + \gamma R_2 + \dots \mid S_0 = s, A_0 = a, \pi],$$

其中 $\gamma \in [0, 1]$ 是一个折扣因子，它权衡即时奖励和后续奖励的重要性。那么最优值就是 $Q_*(s, a) = \max_{\pi} Q_{\pi}(s, a)$ 。通过选择每个状态中价值最高的行动，可以轻松地导出最优策略。

Estimates for the optimal action values can be learned using Q-learning (Watkins, 1989), a form of temporal difference learning (Sutton, 1988). Most interesting problems are too large to learn all action values in all states separately. Instead, we can learn a parameterized value function $Q(s, a; \theta_t)$. The standard Q-learning update for the parameters after taking action A_t in state S_t and observing the immediate reward R_{t+1} and resulting state S_{t+1} is then

$$\theta_{t+1} = \theta_t + \alpha(Y_t^Q - Q(S_t, A_t; \theta_t)) \nabla_{\theta_t} Q(S_t, A_t; \theta_t). \quad (1)$$

where α is a scalar step size and the target Y_t^Q is defined as

$$Y_t^Q \equiv R_{t+1} + \gamma \max_a Q(S_{t+1}, a; \theta_t). \quad (2)$$

This update resembles stochastic gradient descent, updating the current value $Q(S_t, A_t; \theta_t)$ towards a target value Y_t^Q .

Deep Q Networks

A deep Q network (DQN) is a multi-layered neural network that for a given state s outputs a vector of action values $Q(s, \cdot; \theta)$, where θ are the parameters of the network. For an n -dimensional state space and an action space containing m actions, the neural network is a function from $\mathbb{R}^n$ to $\mathbb{R}^m$. Two important ingredients of the DQN algorithm as proposed by Mnih et al. (2015) are the use of a target network, and the use of experience replay. The target network, with parameters θ^- , is the same as the online network except that its parameters are copied every τ steps from the online network, so that then $\theta_t^- = \theta_t$, and kept fixed on all other steps. The target used by DQN is then

$$Y_t^{\text{DQN}} \equiv R_{t+1} + \gamma \max_a Q(S_{t+1}, a; \theta_t^-). \quad (3)$$

For the experience replay (Lin, 1992), observed transitions are stored for some time and sampled uniformly from this memory bank to update the network. Both the target network and the experience replay dramatically improve the performance of the algorithm (Mnih et al., 2015).

Double Q-learning

The max operator in standard Q-learning and DQN, in (2) and (3), uses the same values both to select and to evaluate an action. This makes it more likely to select overestimated values, resulting in overoptimistic value estimates. To prevent this, we can decouple the selection from the evaluation. This is the idea behind Double Q-learning (van Hasselt, 2010).

In the original Double Q-learning algorithm, two value functions are learned by assigning each experience randomly to update one of the two value functions, such that there are two sets of weights, θ and θ' . For each update, one set of weights is used to determine the greedy policy and the other to determine its value. For a clear comparison, we can first untangle the selection and evaluation in Q-learning and rewrite its target (2) as

$$Y_t^Q = R_{t+1} + \gamma Q(S_{t+1}, \arg\max_a Q(S_{t+1}, a; \theta_t); \theta_t).$$

The Double Q-learning error can then be written as

$$Y_t^{\text{DoubleQ}} \equiv R_{t+1} + \gamma Q(S_{t+1}, \arg\max_a Q(S_{t+1}, a; \theta_t); \theta'_t). \quad (4)$$

Notice that the selection of the action, in the $\arg\max$, is still due to the online weights θ_t . This means that, as in Q-learning, we are still estimating the value of the greedy policy according to the current values, as defined by θ_t . However, we use the second set of weights θ'_t to fairly evaluate the value of this policy. This second set of weights can be updated symmetrically by switching the roles of θ and θ' .

Overoptimism due to estimation errors

Q-learning's overestimations were first investigated by Thrun and Schwartz (1993), who showed that if the action values contain random errors uniformly distributed in an interval $[-\epsilon, \epsilon]$ then each target is overestimated up to $\gamma\epsilon\frac{m-1}{m+1}$, where m is the number of actions. In addition, Thrun and Schwartz give a concrete example in which these overestimations even asymptotically lead to sub-optimal policies, and show the overestimations manifest themselves in a small toy problem when using function approximation. Later van Hasselt (2010) argued that noise in the environment can lead to overestimations even when using tabular representation, and proposed Double Q-learning as a solution.

In this section we demonstrate more generally that estimation errors of any kind can induce an upward bias, regardless of whether these errors are due to environmental noise, function approximation, non-stationarity, or any other source. This is important, because in practice any method will incur some inaccuracies during learning, simply due to the fact that the true values are initially unknown.

The result by Thrun and Schwartz (1993) cited above gives an upper bound to the overestimation for a specific setup, but it is also possible, and potentially more interesting, to derive a lower bound.

Theorem 1. *Consider a state s in which all the true optimal action values are equal at $Q_*(s, a) = V_*(s)$ for some $V_*(s)$. Let Q_t be arbitrary value estimates that are on the whole unbiased in the sense that $\sum_a (Q_t(s, a) - V_*(s)) = 0$, but that are not all correct, such that $\frac{1}{m} \sum_a (Q_t(s, a) - V_*(s))^2 = C$ for some $C > 0$, where $m \geq 2$ is the number of actions in s .*

Under these conditions, $\max_a Q_t(s, a) \geq V_(s) + \sqrt{\frac{C}{m-1}}$. This lower bound is tight. Under the same conditions, the lower bound on the absolute error of the Double Q-learning estimate is zero. (Proof in appendix.)*

Note that we did not need to assume that estimation errors for different actions are independent. This theorem shows that even if the value estimates are on average correct, estimation errors of any source can drive the estimates up and away from the true optimal values.

The lower bound in Theorem 1 decreases with the number of actions. This is an artifact of considering the lower bound, which requires very specific values to be attained. More typically, the overoptimism increases with the number of actions as shown in Figure 1. Q-learning's overestimations there indeed increase with the number of actions,

最佳动作值的估计可以使用 Q 学习 (Watkins, 1989) 来学习, Q 学习是时间差分学习的一种形式 (Sutton, 1988)。大多数有趣的问题都太大, 无法单独学习所有状态下的所有动作值。相反, 我们可以学习一个参数化值函数 $Q(s, a; \theta_t)$ 。在状态 S_t 下采取动作 A_t 并观察即时奖励 R_{t+1} 和结果状态 S_{t+1} 后, 参数的标准 Q 学习更新为

$$\theta_{t+1} = \theta_t + \alpha(Y_t^Q - Q(S_t, A_t; \theta_t)) \nabla_{\theta_t} Q(S_t, A_t; \theta_t). \quad (1)$$

其中 α 是标量步长, 目标 Y_t^Q 定义为

$$Y_t^Q \equiv R_{t+1} + \gamma \max_a Q(S_{t+1}, a; \theta_t). \quad (2)$$

这个更新类似于随机梯度下降, 更新当前值 $Q(S_t, A_t; \theta_t)$ 朝向目标值 Y_t^Q 。

深Q网络

深度 Q 网络 (DQN) 是一种多层神经网络, 对于给定状态 s 输出动作值向量 $Q(s, \cdot; \theta)$, 其中 θ 是网络的参数。对于 n 维状态空间和包含 m 动作的动作空间, 神经网络是从 $\mathbb{R}^n$ 到 $\mathbb{R}^m$ 的函数。Mnih 等人提出的 DQN 算法的两个重要组成部分。(2015) 是目标网络的使用和经验回放的使用。具有参数 θ^- 的目标网络与在线网络相同, 只是它的参数从在线网络的每 τ 步骤复制一次, 以便然后是 $\theta_t^- = \theta_t$, 并在所有其他步骤上保持固定。DQN 使用的目标是

$$Y_t^{\text{DQN}} \equiv R_{t+1} + \gamma \max_a Q(S_{t+1}, a; \theta_t^-). \quad (3)$$

对于经验重放 (Lin, 1992), 观察到的转换被存储一段时间, 并从该存储库中统一采样以更新网络。目标网络和经验回放都极大地提高了算法的性能 (Mnih et al., 2015)。

双Q学习

(2) 和 (3) 中标准 Q 学习和 DQN 中的 $\max$ 运算符使用相同的值来选择和评估动作。这使得更有可能选择高估的值, 从而导致过度乐观的价值估计。为了防止这种情况, 我们可以将选择与评估分离。这就是双 Q 学习背后的想法 (van Hasselt, 2010)。

在原始的双 Q 学习算法中, 通过随机分配每个经验来更新两个值函数之一来学习两个值函数, 从而有两组权重, θ 和 θ' 。对于每次更新, 一组权重用于确定贪婪策略, 另一组权重用于确定其值。为了清楚地比较, 我们可以首先理清 Q-learning 中的选择和评估, 并将其目标 (2) 重写为

$$Y_t^Q = R_{t+1} + \gamma Q(S_{t+1}, \arg\max_a Q(S_{t+1}, a; \theta_t); \theta_t).$$

双 Q 学习误差可以写为

$$Y_t^{\text{DoubleQ}} \equiv R_{t+1} + \gamma Q(S_{t+1}, \arg\max_a Q(S_{t+1}, a; \theta_t); \theta'_t). \quad (4)$$

请注意, $\arg\max$ 中操作的选择仍然取决于在线权重 θ_t 。这意味着, 就像在 Q 学习中一样, 我们仍然根据当前值 (由 θ_t 定义) 来估计贪婪策略的值。然而, 我们使用第二组权重 θ'_t 来公平地评估该策略的价值。第二组权重可以通过切换 θ 和 θ' 的角色来对称更新。

由于估计错误而过度乐观

Thrun 和 Schwartz (1993) 首先研究了 Q 学习的高估, 他们表明, 如果动作值包含均匀分布在区间 $[-\epsilon, \epsilon]$ 中的随机误差, 那么每个目标都会被高估到 $\gamma \epsilon \frac{m-1}{m+1}$, 其中 m 是动作的数量。此外, Thrun 和 Schwartz 给出了一个具体的例子, 其中这些高估甚至渐进地导致次优策略, 并表明在使用函数逼近时, 高估会在一个小玩具问题中表现出来。后来 van Hasselt (2010) 认为, 即使使用表格表示, 环境中的噪声也会导致高估, 并提出了双 Q 学习作为解决方案。

在本节中, 我们更普遍地证明, 任何类型的估计误差都可能导致向上偏差, 无论这些误差是由于环境噪声、函数逼近、非平稳性还是任何其他来源造成的。这很重要, 因为在实践中任何方法在学习过程中都会出现一些不准确的情况, 这仅仅是因为真实值最初是未知的。

上面引用的 Thrun 和 Schwartz (1993) 的结果给出了特定设置的高估上限, 但也有可能得出一个下限, 而且可能更有趣。

定理1. Consider a state s in which all the true optimal action values are equal at $Q_*(s, a) = V_*(s)$ for some $V_*(s)$. Let Q_t be arbitrary value estimates that are on the whole unbiased in the sense that $\sum_a (Q_t(s, a) - V_*(s)) = 0$, but that are not all correct, such that $\frac{1}{m} \sum_a (Q_t(s, a) - V_*(s))^2 = C$ for some $C > 0$, where $m \geq 2$ is the number of actions in s . Under these conditions, $\max_a Q_t(s, a) \geq V_*(s) + \sqrt{\frac{C}{m-1}}$. This lower bound is tight. Under the same conditions, the lower bound on the absolute error of the Double Q-learning estimate is zero. (Proof in appendix.)

请注意, 我们不需要假设不同操作的估计误差是独立的。该定理表明, 即使值估计平均而言是正确的, 任何来源的估计误差都可能导致估计值偏离真正的最优值。

定理 1 中的下界随着动作数量的增加而减小。这是考虑下限的产物, 需要达到非常具体的值。更典型的是, 过度乐观随着行动数量的增加而增加, 如图 1 所示。Q-learning 的高估确实随着行动数量的增加而增加,

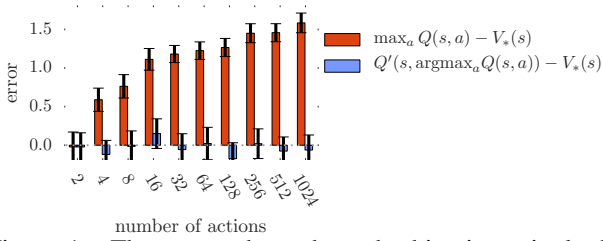


Figure 1: The orange bars show the bias in a single Q-learning update when the action values are $Q(s, a) = V_*(s) + \epsilon_a$ and the errors $\{\epsilon_a\}_{a=1}^m$ are independent standard normal random variables. The second set of action values Q' , used for the blue bars, was generated identically and independently. All bars are the average of 100 repetitions.

while Double Q-learning is unbiased. As another example, if for all actions $Q_*(s, a) = V_*(s)$ and the estimation errors $Q_t(s, a) - V_*(s)$ are uniformly random in $[-1, 1]$, then the overoptimism is $\frac{m-1}{m+1}$. (Proof in appendix.)

We now turn to function approximation and consider a real-valued continuous state space with 10 discrete actions in each state. For simplicity, the true optimal action values in this example depend only on state so that in each state all actions have the same true value. These true values are shown in the left column of plots in Figure 2 (purple lines) and are defined as either $Q_*(s, a) = \sin(s)$ (top row) or $Q_*(s, a) = 2 \exp(-s^2)$ (middle and bottom rows). The left plots also show an approximation for a single action (green lines) as a function of state as well as the samples the estimate is based on (green dots). The estimate is a d -degree polynomial that is fit to the true values at sampled states, where $d = 6$ (top and middle rows) or $d = 9$ (bottom row). The samples match the true function exactly: there is no noise and we assume we have ground truth for the action value on these sampled states. The approximation is inexact even on the sampled states for the top two rows because the function approximation is insufficiently flexible. In the bottom row, the function is flexible enough to fit the green dots, but this reduces the accuracy in unsampled states. Notice that the sampled states are spaced further apart near the left side of the left plots, resulting in larger estimation errors. In many ways this is a typical learning setting, where at each point in time we only have limited data.

The middle column of plots in Figure 2 shows estimated action value functions for all 10 actions (green lines), as functions of state, along with the maximum action value in each state (black dashed line). Although the true value function is the same for all actions, the approximations differ because we have supplied different sets of sampled states.¹ The maximum is often higher than the ground truth shown in purple on the left. This is confirmed in the right plots, which shows the difference between the black and purple curves in orange. The orange line is almost always positive,

¹Each action-value function is fit with a different subset of integer states. States -6 and 6 are always included to avoid extrapolations, and for each action two adjacent integers are missing: for action a_1 states -5 and -4 are not sampled, for a_2 states -4 and -3 are not sampled, and so on. This causes the estimated values to differ.

indicating an upward bias. The right plots also show the estimates from Double Q-learning in blue², which are on average much closer to zero. This demonstrates that Double Q-learning indeed can successfully reduce the overoptimism of Q-learning.

The different rows in Figure 2 show variations of the same experiment. The difference between the top and middle rows is the true value function, demonstrating that overestimations are not an artifact of a specific true value function. The difference between the middle and bottom rows is the flexibility of the function approximation. In the left-middle plot, the estimates are even incorrect for some of the sampled states because the function is insufficiently flexible. The function in the bottom-left plot is more flexible but this causes higher estimation errors for unseen states, resulting in higher overestimations. This is important because flexible parametric function approximators are often employed in reinforcement learning (see, e.g., Tesauro 1995; Sallans and Hinton 2004; Riedmiller 2005; Mnih et al. 2015).

In contrast to van Hasselt (2010) we did not use a statistical argument to find overestimations, the process to obtain Figure 2 is fully deterministic. In contrast to Thrun and Schwartz (1993), we did not rely on inflexible function approximation with irreducible asymptotic errors; the bottom row shows that a function that is flexible enough to cover all samples leads to high overestimations. This indicates that the overestimations can occur quite generally.

In the examples above, overestimations occur even when assuming we have samples of the *true* action value at certain states. The value estimates can further deteriorate if we bootstrap off of action values that are already overoptimistic, since this causes overestimations to propagate throughout our estimates. Although *uniformly* overestimating values might not hurt the resulting policy, in practice overestimation errors will differ for different states and actions. Overestimation combined with bootstrapping then has the pernicious effect of propagating the wrong relative information about which states are more valuable than others, directly affecting the quality of the learned policies.

The overestimations should not be confused with optimism in the face of uncertainty (Sutton, 1990; Agrawal, 1995; Kaelbling et al., 1996; Auer et al., 2002; Brafman and Tennenholtz, 2003; Szita and Lőrincz, 2008; Strehl et al., 2009), where an exploration bonus is given to states or actions with uncertain values. Conversely, the overestimations discussed here occur only after updating, resulting in overoptimism in the face of apparent certainty. This was already observed by Thrun and Schwartz (1993), who noted that, in contrast to optimism in the face of uncertainty, these overestimations actually can impede learning an optimal policy. We will see this negative effect on policy quality confirmed later in the experiments as well: when we reduce the overestimations using Double Q-learning, the policies improve.

²We arbitrarily used the samples of action a_{i+5} (for $i \leq 5$) or a_{i-5} (for $i > 5$) as the second set of samples for the double estimator of action a_i .

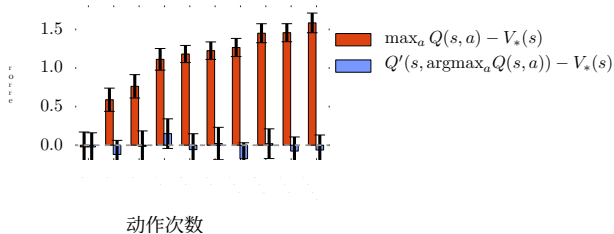


图 1: 橙色条显示当操作值为 $Q(s, a) = V_*(s) + \epsilon_a$ 且误差 $\{\epsilon_a\}_{a=1}^m$ 为独立标准正态随机变量时, 单次 Q 学习更新中的偏差。用于蓝色条的第二组动作值 Q' 是相同且独立生成的。所有条形均为 100 次重复的平均值。

而双 Q 学习是无偏的。作为另一个示例, 如果对于所有动作 $Q_*(s, a) = V_*(s)$ 且估计误差 $Q_t(s, a) - V_*(s)$ 在 $[-1, 1]$ 中均匀随机, 则过度乐观为 $\frac{m-1}{m+1}$ 。(证明见附录。)

现在我们转向函数逼近, 并考虑每个状态有 10 个离散动作的实值连续状态空间。为简单起见, 本例中真正的最佳动作值仅取决于状态, 因此在每个状态中所有动作都具有相同的真实值。这些真值显示在图 2 的左列 (紫色线) 中, 并定义为 $Q_*(s, a) = \sin(s)$ (顶行) 或 $Q_*(s, a) = 2 \exp(-s^2)$ (中行和底行)。左图还显示了作为状态函数的单个动作 (绿线) 的近似值以及估计所基于的样本 (绿点)。估计值是一个 d 次多项式, 它适合采样状态下的真实值, 其中 $d = 6$ (顶部和中间行) 或 $d = 9$ (底部行)。样本与真实函数完全匹配: 没有噪声, 我们假设我们有这些采样状态上的动作值的基本事实。即使在前两行的采样状态上, 近似也是不精确的, 因为函数近似不够灵活。在底行中, 该函数足够灵活, 可以适应绿点, 但这会降低未采样状态下的准确性。请注意, 采样状态在左图左侧附近间隔更远, 导致更大的估计误差。从很多方面来说, 这是一个典型的学习环境, 在每个时间点我们只有有限的数据。

图 2 中的中间一列图显示了所有 10 个动作的估计动作值函数 (绿线), 作为状态函数, 以及每个状态的最大动作值 (黑色虚线)。尽管所有动作的真值函数都是相同的, 但近似值有所不同, 因为我们提供了不同的采样状态集。¹最大值通常高于左侧紫色所示的基本事实。这在右图中得到了证实, 右图中以橙色显示了黑色和紫色曲线之间的差异。橙色线几乎总是正极,

¹Each action-value function is fit with a different subset of integer states. States -6 and 6 are always included to avoid extrapolations, and for each action two adjacent integers are missing: for action a_1 states -5 and -4 are not sampled, for a_2 states -4 and -3 are not sampled, and so on. This causes the estimated values to differ.

表明存在向上的倾向。右图还显示了蓝色²中双 Q 学习的估计值, 平均而言更接近于零。这表明 Double Q-learning 确实可以成功地减少 Q-learning 的过度乐观。

图 2 中的不同行显示了同一实验的变化。顶行和中间行之间的差异是真值函数, 这表明高估并不是特定真值函数的人为因素。中行和底行之间的区别在于函数逼近的灵活性。在左图中, 由于函数不够灵活, 某些采样状态的估计甚至不正确。左下图中的函数更加灵活, 但这会导致对未见状态的估计误差更高, 从而导致更高的高估。这很重要, 因为强化学习中经常采用灵活的参数函数逼近器 (参见 Tesauro 1995; Sallans 和 Hinton 2004; Riedmiller 2005; Mnih 等人 2015)。

与 van Hasselt (2010) 相比, 我们没有使用统计论证来发现高估, 获得图 2 的过程是完全确定性的。与 Thrun 和 Schwartz (1993) 相比, 我们不依赖具有不可约渐近误差的僵化函数逼近; 底行显示, 足够灵活以覆盖所有样本的函数会导致严重高估。这表明高估现象可能相当普遍地发生。

在上面的例子中, 即使假设我们有某些状态下的 *true* 动作值样本, 也会发生高估。如果从已经过于乐观的行动值中引导, 价值估计可能会进一步恶化, 因为这会导致高估在我们的估计中传播。尽管 *uniformly* 高估值可能不会损害最终的策略, 但实际上, 不同状态和操作的高估错误会有所不同。过度估计与自举法相结合会产生有害的影响, 即传播关于哪些状态比其他状态更有价值的错误相关信息, 直接影响学习政策的质量。

面对不确定性, 高估不应与乐观相混淆 (Sutton, 1990; Agrawal, 1995; Kaelbling 等, 1996; Auer 等, 2002; Brafman 和 Tennenholtz, 2003; Szita 和 Lőrincz, 2008; Strehl 等, 2009), 其中探索奖励给予具有不确定性的状态或动作。相反, 这里讨论的高估仅发生在更新之后, 导致面对明显的确定性时过度乐观。Thrun 和 Schwartz (1993) 已经观察到了这一点, 他们指出, 与面对不确定性时的乐观情绪相反, 这些高估实际上会阻碍学习最优策略。我们将在稍后的实验中看到这种对政策质量的负面影响: 当我们使用双 Q 学习减少高估时, 政策会得到改善。

²We arbitrarily used the samples of action a_{i+5} (for $i \leq 5$) or a_{i-5} (for $i > 5$) as the second set of samples for the double estimator of action a_i .

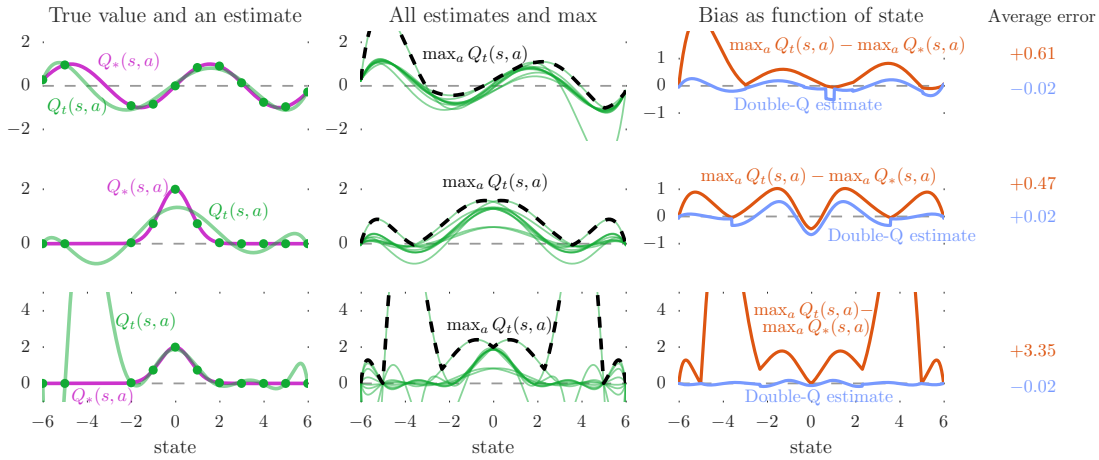


Figure 2: Illustration of overestimations during learning. In each state (x-axis), there are 10 actions. The **left column** shows the true values $V_*(s)$ (purple line). All true action values are defined by $Q_*(s, a) = V_*(s)$. The green line shows estimated values $Q(s, a)$ for one action as a function of state, fitted to the true value at several sampled states (green dots). The **middle column** plots show all the estimated values (green), and the maximum of these values (dashed black). The maximum is higher than the true value (purple, left plot) almost everywhere. The **right column** plots shows the difference in orange. The blue line in the right plots is the estimate used by Double Q-learning with a second set of samples for each state. The blue line is much closer to zero, indicating less bias. The three **rows** correspond to different true functions (left, purple) or capacities of the fitted function (left, green). (Details in the text)

Double DQN

The idea of Double Q-learning is to reduce overestimations by decomposing the max operation in the target into action selection and action evaluation. Although not fully decoupled, the target network in the DQN architecture provides a natural candidate for the second value function, without having to introduce additional networks. We therefore propose to evaluate the greedy policy according to the online network, but using the target network to estimate its value. In reference to both Double Q-learning and DQN, we refer to the resulting algorithm as Double DQN. Its update is the same as for DQN, but replacing the target Y_t^{DQN} with

$$Y_t^{\text{DoubleDQN}} \equiv R_{t+1} + \gamma Q(S_{t+1}, \arg\max_a Q(S_{t+1}, a; \theta_t), \theta_t^-).$$

In comparison to Double Q-learning (4), the weights of the second network θ_t' are replaced with the weights of the target network θ_t^- for the evaluation of the current greedy policy. The update to the target network stays unchanged from DQN, and remains a periodic copy of the online network.

This version of Double DQN is perhaps the minimal possible change to DQN towards Double Q-learning. The goal is to get most of the benefit of Double Q-learning, while keeping the rest of the DQN algorithm intact for a fair comparison, and with minimal computational overhead.

Empirical results

In this section, we analyze the overestimations of DQN and show that Double DQN improves over DQN both in terms of value accuracy and in terms of policy quality. To further test the robustness of the approach we additionally evaluate the algorithms with random starts generated from expert human trajectories, as proposed by Nair et al. (2015).

Our testbed consists of Atari 2600 games, using the Arcade Learning Environment (Bellemare et al., 2013). The

goal is for a single algorithm, with a fixed set of hyperparameters, to learn to play each of the games separately from interaction given only the screen pixels as input. This is a demanding testbed: not only are the inputs high-dimensional, the game visuals and game mechanics vary substantially between games. Good solutions must therefore rely heavily on the learning algorithm — it is not practically feasible to overfit the domain by relying only on tuning.

We closely follow the experimental setting and network architecture outlined by Mnih et al. (2015). Briefly, the network architecture is a convolutional neural network (Fukushima, 1988; LeCun et al., 1998) with 3 convolution layers and a fully-connected hidden layer (approximately 1.5M parameters in total). The network takes the last four frames as input and outputs the action value of each action. On each game, the network is trained on a single GPU for 200M frames, or approximately 1 week.

Results on overoptimism

Figure 3 shows examples of DQN’s overestimations in six Atari games. DQN and Double DQN were both trained under the exact conditions described by Mnih et al. (2015). DQN is consistently and sometimes vastly overoptimistic about the value of the current greedy policy, as can be seen by comparing the orange learning curves in the top row of plots to the straight orange lines, which represent the actual discounted value of the best learned policy. More precisely, the (averaged) value estimates are computed regularly during training with full evaluation phases of length $T = 125,000$ steps as

$$\frac{1}{T} \sum_{t=1}^T \arg\max_a Q(S_t, a; \theta).$$

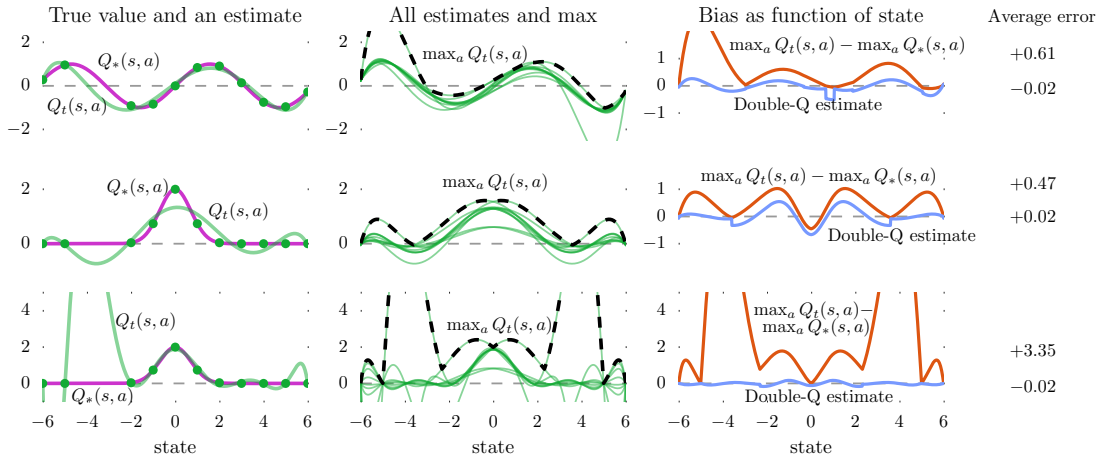


图 2: 学习过程中高估的图示。在每个状态 (x 轴) 中, 有 10 个操作。左栏显示真实值 $V_*(s)$ (紫色线)。所有真实的操作值均由 $Q_*(s, a) = V_*(s)$ 定义。绿线显示了作为状态函数的一个动作的估计值 $Q_t(s, a)$, 与几个采样状态 (绿点) 的真实值相拟合。中间的柱图显示所有估计值 (绿色) 以及这些值的最大值 (黑色虚线)。几乎所有地方的最大值都高于真实值 (紫色, 左图)。右柱图显示了橙色的差异。右图中的蓝线是双 Q 学习使用的估计值, 每个状态都有第二组样本。蓝线更接近于零, 表明偏差较小。三行对应于不同的真实函数 (左, 紫色) 或拟合函数的容量 (左, 绿色)。(详情见文中)

双DQN

双Q学习的思想是通过将目标中的最大操作分解为动作选择和动作评估来减少高估。尽管没有完全解耦, DQN 架构中的目标网络为第二个值函数提供了自然的候选者, 而无需引入额外的网络。因此, 我们建议根据在线网络来评估贪婪策略, 但使用目标网络来估计其价值。对于 Double Q-learning 和 DQN, 我们将生成的算法称为 Double DQN。其更新与 DQN 相同, 但将目标 Y_t^{DQN} 替换为

$$Y_t^{\text{DoubleDQN}} \equiv R_{t+1} + \gamma Q(S_{t+1}, \arg\max_a Q(S_{t+1}, a; \theta_t), \theta_t^-).$$

与双Q学习 (4) 相比, 第二个网络 θ_t' 的权重被目标网络 θ_t^- 的权重替换, 以评估当前的贪婪策略。对目标网络的更新与 DQN 保持不变, 并且仍然是在线网络的定期副本。

Double DQN 的这个版本可能是 DQN 向 Double Q-learning 的最小可能改变。目标是获得双 Q 学习的大部分好处, 同时保持 DQN 算法的其余部分完整以进行公平比较, 并以最小的计算开销。

实证结果

在本节中, 我们分析了 DQN 的高估, 并表明 Double DQN 在价值准确性和策略质量方面均优于 DQN。为了进一步测试该方法的稳健性, 我们还使用从专家人类轨迹生成的随机启动来评估算法, 正如 Nair 等人提出的那样。(2015)。

我们的测试平台由 Atari 2600 游戏组成, 使用 Arcade 学习环境 (Bellemare 等, 2013)。这

目标是使用具有一组固定超参数的单一算法, 在仅以屏幕像素作为输入的情况下, 学习独立于交互来玩每个游戏。这是一个要求很高的测试平台: 不仅输入是高维的, 游戏之间的游戏视觉效果和游戏机制也有很大差异。因此, 好的解决方案必须严重依赖学习算法——仅依靠调整来过度拟合域实际上是不可行的。

我们密切关注 Mnih 等人概述的实验设置和网络架构。(2015)。简而言之, 网络架构是一个卷积神经网络 (Fukushima, 1988; LeCun 等人, 1998), 具有 3 个卷积层和一个全连接隐藏层 (总共大约 150 万个参数)。网络将最后四帧作为输入, 输出每个动作的动作值。在每款游戏中, 网络都会在单个 GPU 上进行 2 亿帧的训练, 即大约 1 周。

过度乐观的结果

图 3 显示了 DQN 在六款 Atari 游戏中高估的示例。DQN 和 Double DQN 均在 Mnih 等人描述的精确条件下进行训练。(2015)。DQN 对当前贪婪策略的价值始终过于乐观, 有时甚至过于乐观, 通过将图中顶行中的橙色学习曲线与橙色直线进行比较可以看出, 橙色直线代表最佳学习策略的实际贴现值。更准确地说, 在训练期间定期计算 (平均) 值估计, 长度为 $T = 125,000$ 步的完整评估阶段为

$$\frac{1}{T} \sum_{t=1}^T \arg\max_a Q(S_t, a; \theta).$$

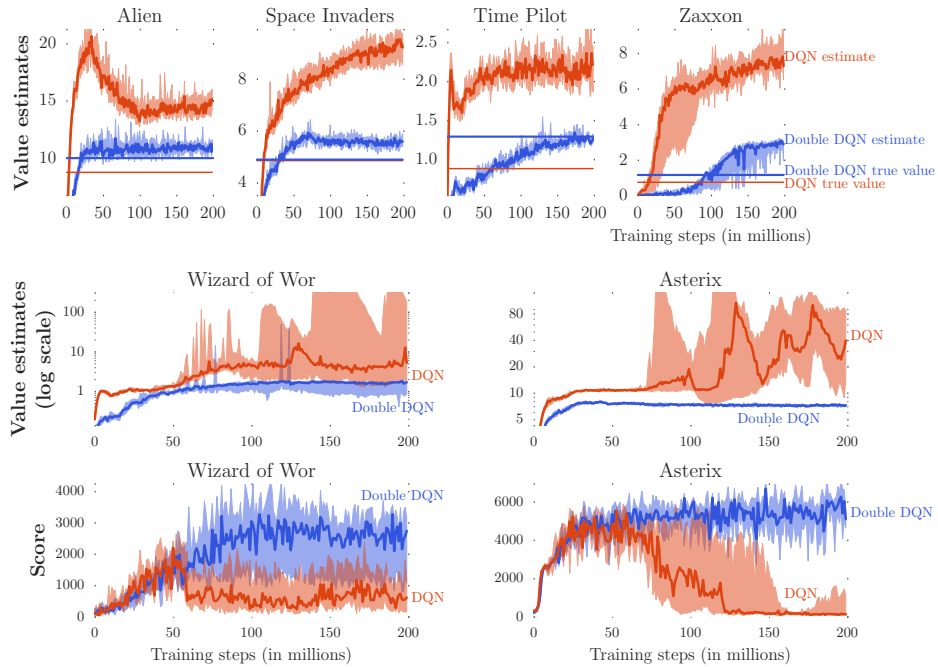


Figure 3: The **top** and **middle** rows show value estimates by DQN (orange) and Double DQN (blue) on six Atari games. The results are obtained by running DQN and Double DQN with 6 different random seeds with the hyper-parameters employed by Mnih et al. (2015). The darker line shows the median over seeds and we average the two extreme values to obtain the shaded area (i.e., 10% and 90% quantiles with linear interpolation). The straight horizontal orange (for DQN) and blue (for Double DQN) lines in the top row are computed by running the corresponding agents after learning concluded, and averaging the actual discounted return obtained from each visited state. These straight lines would match the learning curves at the right side of the plots if there is no bias. The **middle** row shows the value estimates (in log scale) for two games in which DQN’s overoptimism is quite extreme. The **bottom** row shows the detrimental effect of this on the score achieved by the agent as it is evaluated during training: the scores drop when the overestimations begin. Learning with Double DQN is much more stable.

The ground truth averaged values are obtained by running the best learned policies for several episodes and computing the actual cumulative rewards. Without overestimations we would expect these quantities to match up (i.e., the curve to match the straight line at the right of each plot). Instead, the learning curves of DQN consistently end up much higher than the true values. The learning curves for Double DQN, shown in blue, are much closer to the blue straight line representing the true value of the final policy. Note that the blue straight line is often higher than the orange straight line. This indicates that Double DQN does not just produce more accurate value estimates but also better policies.

More extreme overestimations are shown in the middle two plots, where DQN is highly unstable on the games Asterix and Wizard of Wor. Notice the log scale for the values on the y -axis. The bottom two plots shows the corresponding scores for these two games. Notice that the increases in value estimates for DQN in the middle plots coincide with decreasing scores in bottom plots. Again, this indicates that the overestimations are harming the quality of the resulting policies. If seen in isolation, one might perhaps be tempted to think the observed instability is related to inherent instability problems of off-policy learning with function approximation (Baird, 1995; Tsitsiklis and Van Roy, 1997; Sutton et al., 2008; Maei, 2011; Sutton et al., 2015). However, we see that learning is much more stable with Double DQN,

	DQN	Double DQN
Median	93.5%	114.7%
Mean	241.1%	330.3%

Table 1: Summary of normalized performance up to 5 minutes of play on 49 games. Results for DQN are from Mnih et al. (2015)

suggesting that the cause for these instabilities is in fact Q-learning’s overoptimism. Figure 3 only shows a few examples, but overestimations were observed for DQN in all 49 tested Atari games, albeit in varying amounts.

Quality of the learned policies

Overoptimism does not always adversely affect the quality of the learned policy. For example, DQN achieves optimal behavior in Pong despite slightly overestimating the policy value. Nevertheless, reducing overestimations can significantly benefit the stability of learning; we see clear examples of this in Figure 3. We now assess more generally how much Double DQN helps in terms of policy quality by evaluating on all 49 games that DQN was tested on.

As described by Mnih et al. (2015) each evaluation episode starts by executing a special no-op action that does not affect the environment up to 30 times, to provide different starting points for the agent. Some exploration during evaluation provides additional randomization. For Double DQN we used the exact same hyper-parameters as for DQN,

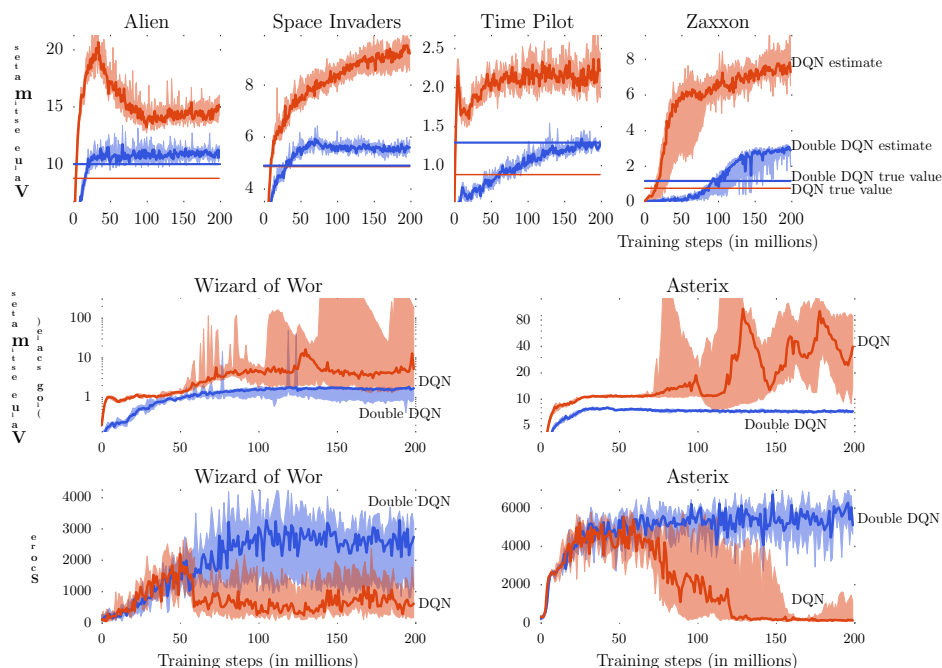


图 3: 顶部和中间行显示 DQN (橙色) 和 Double DQN (蓝色) 对六款 Atari 游戏的价值估计。结果是通过使用 6 个不同的随机种子以及 Mnih 等人采用的超参数运行 DQN 和 Double DQN 获得的。(2015)。深色线显示种子的中位数, 我们对两个极值进行平均以获得阴影面积 (即使用线性插值的 10% 和 90% 分位数)。顶行中的水平直线橙色 (对于 DQN) 和蓝色 (对于双 DQN) 是通过在学习结束后运行相应的代理并对从每个访问状态获得的实际折扣回报进行平均来计算的。如果没有偏差, 这些直线将与图右侧的学习曲线相匹配。中间行显示了 DQN 过度乐观相当极端的两款游戏的价值估计 (以对数标度表示)。底行显示了这对智能体在训练期间评估时获得的分数的不利影响: 当高估开始时, 分数会下降。使用 Double DQN 学习更加稳定。

通过运行几个情节的最佳学习策略并计算实际的累积奖励来获得地面真实平均值。如果没有高估, 我们预计这些数量会匹配 (即曲线与每个图右侧的直线匹配)。相反, DQN 的学习曲线始终远高于真实值。双 DQN 的学习曲线 (以蓝色显示) 更接近代表最终策略真实值的蓝色直线。请注意, 蓝色直线通常高于橙色直线。这表明 Double DQN 不仅能产生更准确的价值估计, 还能产生更好的政策。

中间两张图显示了更极端的高估, 其中 DQN 在 Asterix 和 Wizard of Wor 游戏中高度不稳定。请注意 y 轴上的值的对数刻度。下面两幅图显示了这两款游戏的相应得分。请注意, 中间图中 DQN 值估计值的增加与底部图中分数的下降一致。这再次表明高估正在损害最终政策的质量。如果孤立地看待, 人们可能会认为观察到的不稳定性与函数逼近的离策略学习固有的不稳定性问题有关 (Baird, 1995; Tsitsiklis 和 Van Roy, 1997; Sutton 等人, 2008; Maei, 2011; Sutton 等人, 2015)。然而, 我们看到使用 Double DQN 的学习更加稳定,

	DQN	Double DQN
Median	93.5%	114.7%
Mean	241.1%	330.3%

表 1: 49 场比赛的 5 分钟内标准化性能总结。DQN 的结果来自 Mnih 等人。(2015)

表明这些不稳定的原因实际上是 Q-learning 的过度乐观。图 3 仅显示了几个示例, 但在所有 49 款测试的 Atari 游戏中都观察到 DQN 被高估, 尽管程度不同。

所学政策的质量

过度乐观并不总是会对学习政策的质量产生不利影响。例如, 尽管稍微高估了策略价值, DQN 在 Pong 中实现了最优行为。然而, 减少高估可以显著有利于学习的稳定性。我们在图 3 中看到了这方面的清晰示例。现在, 我们通过评估 DQN 测试的所有 49 款游戏, 更全面地评估 Double DQN 对策略质量的帮助程度。

正如 Mnih 等人所描述的。(2015) 每个评估阶段都从执行不影响环境的特殊无操作操作开始, 最多执行 30 次, 为代理提供不同的起点。评估期间的一些探索提供了额外的随机化。对于 Double DQN, 我们使用与 DQN 完全相同的超参数,

	DQN	Double DQN	Double DQN (tuned)
Median	47.5%	88.4%	116.7%
Mean	122.0%	273.1%	475.2%

Table 2: Summary of normalized performance up to 30 minutes of play on 49 games with human starts. Results for DQN are from Nair et al. (2015).

to allow for a controlled experiment focused just on reducing overestimations. The learned policies are evaluated for 5 mins of emulator time (18,000 frames) with an ϵ -greedy policy where $\epsilon = 0.05$. The scores are averaged over 100 episodes. The only difference between Double DQN and DQN is the target, using $Y_t^{\text{DoubleDQN}}$ rather than Y_t^{DQN} . This evaluation is somewhat adversarial, as the used hyperparameters were tuned for DQN but not for Double DQN.

To obtain summary statistics across games, we normalize the score for each game as follows:

$$\text{score}_{\text{normalized}} = \frac{\text{score}_{\text{agent}} - \text{score}_{\text{random}}}{\text{score}_{\text{human}} - \text{score}_{\text{random}}}. \quad (5)$$

The ‘random’ and ‘human’ scores are the same as used by Mnih et al. (2015), and are given in the appendix.

Table 1, under **no ops**, shows that on the whole Double DQN clearly improves over DQN. A detailed comparison (in appendix) shows that there are several games in which Double DQN greatly improves upon DQN. Noteworthy examples include Road Runner (from 233% to 617%), Asterix (from 70% to 180%), Zaxxon (from 54% to 111%), and Double Dunk (from 17% to 397%).

The Gorila algorithm (Nair et al., 2015), which is a massively distributed version of DQN, is not included in the table because the architecture and infrastructure is sufficiently different to make a direct comparison unclear. For completeness, we note that Gorila obtained median and mean normalized scores of 96% and 495%, respectively.

Robustness to Human starts

One concern with the previous evaluation is that in deterministic games with a unique starting point the learner could potentially learn to remember sequences of actions without much need to generalize. While successful, the solution would not be particularly robust. By testing the agents from various starting points, we can test whether the found solutions generalize well, and as such provide a challenging testbed for the learned policies (Nair et al., 2015).

We obtained 100 starting points sampled for each game from a human expert’s trajectory, as proposed by Nair et al. (2015). We start an evaluation episode from each of these starting points and run the emulator for up to 108,000 frames (30 mins at 60Hz including the trajectory before the starting point). Each agent is only evaluated on the rewards accumulated after the starting point.

For this evaluation we include a tuned version of Double DQN. Some tuning is appropriate because the hyperparameters were tuned for DQN, which is a different algorithm. For the tuned version of Double DQN, we increased the number of frames between each two copies of the target network from 10,000 to 30,000, to reduce overestimations further because immediately after each switch DQN and Double DQN

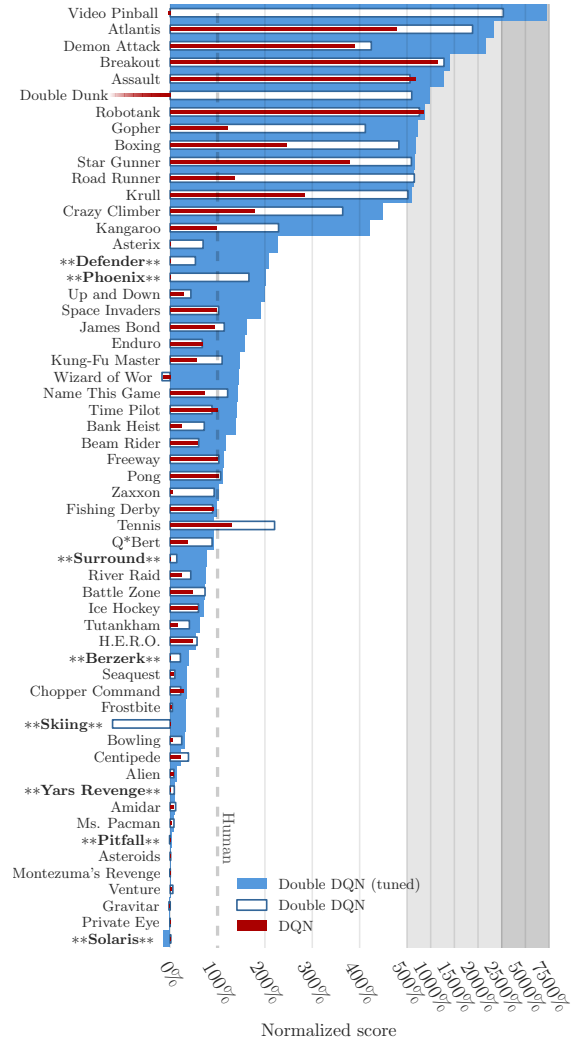


Figure 4: Normalized scores on 57 Atari games, tested for 100 episodes per game with human starts. Compared to Mnih et al. (2015), eight games additional games were tested. These are indicated with stars and a bold font.

both revert to Q-learning. In addition, we reduced the exploration during learning from $\epsilon = 0.1$ to $\epsilon = 0.01$, and then used $\epsilon = 0.001$ during evaluation. Finally, the tuned version uses a single shared bias for all action values in the top layer of the network. Each of these changes improved performance and together they result in clearly better results.³

Table 2 reports summary statistics for this evaluation on the 49 games from Mnih et al. (2015). Double DQN obtains clearly higher median and mean scores. Again Gorila DQN (Nair et al., 2015) is not included in the table, but for completeness note it obtained a median of 78% and a mean of 259%. Detailed results, plus results for an additional 8 games, are available in Figure 4 and in the appendix. On several games the improvements from DQN to Double DQN are striking, in some cases bringing scores much closer to

³Except for Tennis, where the lower ϵ during training seemed to hurt rather than help.

	DQN	Double DQN	Double DQN (tuned)
Median	47.5%	88.4%	116.7%
Mean	122.0%	273.1%	475.2%

表 2: 49 场人类先发比赛的 30 分钟内标准化表现汇总。DQN 的结果来自 Nair 等人。(2015)。

允许进行仅关注减少高估的受控实验。使用 ϵ - 贪婪策略（其中 $\epsilon = 0.05$ ）对学习到的策略进行 5 分钟模拟器时间（18,000 帧）的评估。平均分超过 100 集。Double DQN 和 DQN 之间的唯一区别是目标，使用 $Y_t^{\text{DoubleDQN}}$ 而不是 Y_t^{DQN} 。这种评估有些对抗性，因为所使用的超参数是针对 DQN 进行调整的，而不是针对 Double DQN。

为了获得跨游戏的汇总统计数据，我们将每场游戏的得分标准化如下：

$$\text{score}_{\text{normalized}} = \frac{\text{score}_{\text{agent}} - \text{score}_{\text{random}}}{\text{score}_{\text{human}} - \text{score}_{\text{random}}}. \quad (5)$$

“随机”和“人类”分数与 Mnih 等人使用的相同。(2015)，并在附录中给出。

表 1 在没有操作的情况下显示，总体而言，Double DQN 明显优于 DQN。详细的比较（见附录）表明，有几款 Double DQN 比 DQN 有了很大的改进。值得注意的例子包括 Road Runner（从 233% 到 617%）、Asterix（从 70% 到 180%）、Zaxxon（从 54% 到 111%）和 Double Dunk（从 17% 到 397%）。

Gorila 算法（Nair 等人，2015）是 DQN 的大规模分布式版本，未包含在表中，因为架构和基础设施差异很大，无法进行直接比较。为了完整起见，我们注意到大猩猩的中位数和平均归一化分数分别为 96% 和 495%。

对人类的鲁棒性开始

对先前评估的一个担忧是，在具有独特起点的确定性游戏中，学习者可能会学会记住动作序列，而无需太多概括。虽然该解决方案很成功，但并不是特别稳健。通过从不同的起点测试代理，我们可以测试找到的解决方案是否具有良好泛化性，从而为学习的策略提供具有挑战性的测试平台（Nair et al., 2015）。

正如 Nair 等人提出的那样，我们从人类专家的轨迹中获得了每场比赛的 100 个起点。(2015)。我们从每个起点开始一个评估阶段，并运行模拟器最多 108,000 帧（60Hz 下 30 分钟，包括起点之前的轨迹）。每个智能体仅根据起点后累积的奖励进行评估。

对于此评估，我们包括 Double DQN 的调整版本。一些调整是适当的，因为超参数是针对 DQN 进行调整的，这是一种不同的算法。对于 Double DQN 的调整版本，我们将目标网络的每两个副本之间的帧数从 10,000 增加到 30,000，以进一步减少高估，因为在每次切换 DQN 和 Double DQN 后立即

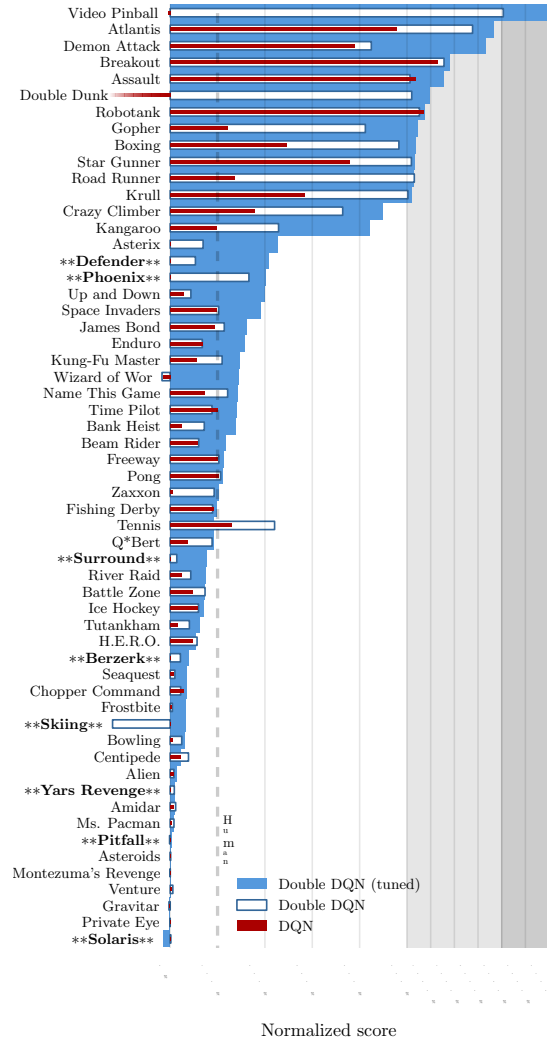


图 4: 57 款 Atari 游戏的归一化得分，每场游戏测试 100 集，并由人类开始。与 Mnih 等人相比。(2015)，测试了八款附加游戏。这些用星号和粗体表示。

两者都恢复到 Q-learning。此外，我们将学习过程中的探索从 $\epsilon = 0.1$ 减少到 $\epsilon = 0.01$ ，然后在评估过程中使用 $\epsilon = 0.001$ 。最后，调整后的版本对网络顶层的所有操作值使用单一共享偏差。这些变化中的每一项都提高了性能，并且它们共同带来了明显更好的结果。³

表 2 报告了 Mnih 等人对 49 场比赛进行评估的汇总统计数据。(2015)。双 DQN 获得明显更高的中位数和平均分。同样，Gorila DQN（Nair 等人，2015）未包含在表中，但为了完整起见，它获得了 78% 的中位数和 259% 的平均值。详细结果以及另外 8 场比赛的结果请参见图 4 和附录。在一些游戏中，从 DQN 到 Double DQN 的改进是惊人的，在某些情况下，得分更接近

³Except for Tennis, where the lower ϵ during training seemed to hurt rather than help.

human, or even surpassing these.

Double DQN appears more robust to this more challenging evaluation, suggesting that appropriate generalizations occur and that the found solutions do not exploit the determinism of the environments. This is appealing, as it indicates progress towards finding general solutions rather than a deterministic sequence of steps that would be less robust.

Discussion

This paper has five contributions. First, we have shown why Q-learning can be overoptimistic in large-scale problems, even if these are deterministic, due to the inherent estimation errors of learning. Second, by analyzing the value estimates on Atari games we have shown that these overestimations are more common and severe in practice than previously acknowledged. Third, we have shown that Double Q-learning can be used at scale to successfully reduce this overoptimism, resulting in more stable and reliable learning. Fourth, we have proposed a specific implementation called Double DQN, that uses the existing architecture and deep neural network of the DQN algorithm without requiring additional networks or parameters. Finally, we have shown that Double DQN finds better policies, obtaining new state-of-the-art results on the Atari 2600 domain.

Acknowledgments

We would like to thank Tom Schaul, Volodymyr Mnih, Marc Bellemare, Thomas Degris, Georg Ostrovski, and Richard Sutton for helpful comments, and everyone at Google DeepMind for a constructive research environment.

References

- R. Agrawal. Sample mean based index policies with $O(\log n)$ regret for the multi-armed bandit problem. *Advances in Applied Probability*, pages 1054–1078, 1995.
- P. Auer, N. Cesa-Bianchi, and P. Fischer. Finite-time analysis of the multiarmed bandit problem. *Machine learning*, 47(2-3):235–256, 2002.
- L. Baird. Residual algorithms: Reinforcement learning with function approximation. In *Machine Learning: Proceedings of the Twelfth International Conference*, pages 30–37, 1995.
- M. G. Bellemare, Y. Naddaf, J. Veness, and M. Bowling. The arcade learning environment: An evaluation platform for general agents. *J. Artif. Intell. Res. (JAIR)*, 47:253–279, 2013.
- R. I. Brafman and M. Tennenholtz. R-max-a general polynomial time algorithm for near-optimal reinforcement learning. *The Journal of Machine Learning Research*, 3:213–231, 2003.
- K. Fukushima. Neocognitron: A hierarchical neural network capable of visual pattern recognition. *Neural networks*, 1(2):119–130, 1988.
- L. P. Kaelbling, M. L. Littman, and A. W. Moore. Reinforcement learning: A survey. *Journal of Artificial Intelligence Research*, 4:237–285, 1996.
- Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- L. Lin. Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine learning*, 8(3):293–321, 1992.
- H. R. Maei. *Gradient temporal-difference learning algorithms*. PhD thesis, University of Alberta, 2011.
- V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.
- A. Nair, P. Srinivasan, S. Blackwell, C. Alcicek, R. Fearon, A. D. Maria, V. Panneershelvam, M. Suleyman, C. Beattie, S. Petersen, S. Legg, V. Mnih, K. Kavukcuoglu, and D. Silver. Massively parallel methods for deep reinforcement learning. In *Deep Learning Workshop, ICML*, 2015.
- M. Riedmiller. Neural fitted Q iteration - first experiences with a data efficient neural reinforcement learning method. In J. Gama, R. Camacho, P. Brazdil, A. Jorge, and L. Torgo, editors, *Proceedings of the 16th European Conference on Machine Learning (ECML’05)*, pages 317–328. Springer, 2005.
- B. Sallans and G. E. Hinton. Reinforcement learning with factored states and actions. *The Journal of Machine Learning Research*, 5:1063–1088, 2004.
- A. L. Strehl, L. Li, and M. L. Littman. Reinforcement learning in finite MDPs: PAC analysis. *The Journal of Machine Learning Research*, 10:2413–2444, 2009.
- R. S. Sutton. Learning to predict by the methods of temporal differences. *Machine learning*, 3(1):9–44, 1988.
- R. S. Sutton. Integrated architectures for learning, planning, and reacting based on approximating dynamic programming. In *Proceedings of the seventh international conference on machine learning*, pages 216–224, 1990.
- R. S. Sutton and A. G. Barto. *Introduction to reinforcement learning*. MIT Press, 1998.
- R. S. Sutton, C. Szepesvári, and H. R. Maei. A convergent $O(n)$ algorithm for off-policy temporal-difference learning with linear function approximation. *Advances in Neural Information Processing Systems 21 (NIPS-08)*, 21:1609–1616, 2008.
- R. S. Sutton, A. R. Mahmood, and M. White. An emphatic approach to the problem of off-policy temporal-difference learning. *arXiv preprint arXiv:1503.04269*, 2015.
- I. Szita and A. Lőrincz. The many faces of optimism: a unifying approach. In *Proceedings of the 25th international conference on Machine learning*, pages 1048–1055. ACM, 2008.
- G. Tesauro. Temporal difference learning and td-gammon. *Communications of the ACM*, 38(3):58–68, 1995.
- S. Thrun and A. Schwartz. Issues in using function approximation for reinforcement learning. In M. Mozer, P. Smolensky, D. Touretzky, J. Elman, and A. Weigend, editors, *Proceedings of the 1993 Connectionist Models Summer School*, Hillsdale, NJ, 1993. Lawrence Erlbaum.
- J. N. Tsitsiklis and B. Van Roy. An analysis of temporal-difference learning with function approximation. *IEEE Transactions on Automatic Control*, 42(5):674–690, 1997.
- H. van Hasselt. Double Q-learning. *Advances in Neural Information Processing Systems*, 23:2613–2621, 2010.
- H. van Hasselt. *Insights in Reinforcement Learning*. PhD thesis, Utrecht University, 2011.
- C. J. C. H. Watkins. *Learning from delayed rewards*. PhD thesis, University of Cambridge England, 1989.

人类，甚至超越这些。

双 DQN 对于这种更具挑战性的评估似乎更稳健，这表明出现了适当的概括，并且找到的解决方案没有利用环境的决定论。这很有吸引力，因为它表明在寻找通用解决方案方面取得了进展，而不是稳健性较差的确定性步骤序列。

讨论

本文有五个贡献。首先，我们展示了为什么 Q 学习在大规模问题中可能过于乐观，即使这些问题是确定性的，因为学习固有的估计误差。其次，通过分析雅达利游戏的价值估计，我们发现这些高估在实践中比以前承认的更为普遍和严重。第三，我们已经证明，可以大规模使用双 Q 学习来成功减少这种过度乐观，从而实现更加稳定和可靠的学习。第四，我们提出了一种称为 Double DQN 的具体实现，它使用 DQN 算法的现有架构和深度神经网络，而不需要额外的网络或参数。最后，我们证明了 Double DQN 找到了更好的策略，在 Atari 2600 域上获得了新的最先进的结果。

致谢

我们要感谢 Tom Schaul、Volodymyr Mnih、Marc Bellemare、Thomas Degris、Georg Ostrovski 和 Richard Sutton 提供的有益评论，并感谢 Google DeepMind 的每个人提供的建设性研究环境。

参考

R.阿格拉瓦尔。针对多臂老虎机问题，基于平均值的索引策略样本具有 $O(\log n)$ 遗憾。 *Advances in Applied Probability*, 第 1054–1078 页, 1995 年。P. Auer、N. Cesa-Bianchi 和 P. Fischer。多臂老虎机问题的有限时间分析。 *Machine learning*, 47(2-3):235–256, 2002。L.贝尔德。残差算法：具有函数逼近的强化学习。 *Machine Learning: Proceedings of the Twelfth International Conference*, 第 30-37 页, 1995 年。M. G. Bellemare、Y. Naddaf、J. Veness 和 M. Bowling。街机学习环境：通用智能体的评估平台。 *J. Artif. Intell. Res. (JAIR)*, 47: 253–279, 2013。R.I.布拉夫曼和 M.Tennenholtz。R-max - 用于接近最优强化学习的通用多项式时间算法。 *The Journal of Machine Learning Research*, 3:213–231, 2003。K.福岛。Neocognitron：一种能够进行视觉模式识别的分层神经网络。 *Neural networks*, 1(2):119–130, 1988。L.P.Kaelbling、M.L.Littman 和 A.W.Moore。强化学习：一项调查。 *Journal of Artificial Intelligence Research*, 4:237–285, 1996。Y. LeCun、L. Bottou、Y. Bengio 和 P. Haffner。基于梯度的学习应用于文档识别。 *Proceedings of the IEEE*, 86(11):2278–2324, 1998。L.林。基于强化学习、规划和教学的自我改进反应代理。 *Machine learning*, 8 (3) : 293-321, 1992。

H.R.梅伊。 *Gradient temporal-difference learning algorithms*。博士论文，阿尔伯塔大学，2011 年。V. Mnih、K. Kavukcuoglu、D. Silver、A. A. Rusu、J. Veness、M. G. Bellemare、A. Graves、M. Riedmiller、A. K. Fidjeland、G. Ostrovski、S. Petersen、C. Beattie、A. Sadik、I. Antonoglou、H. King、D. Kumaran、D. Wierstra、S. Legg 和 D. Hassabis。通过深度强化学习进行人类水平的控制。 *Nature*, 518 (7540):529–533, 2015。A. Nair、P. Srinivasan、S. Blackwell、C. Alcicek、R. Fearon、A. D. Maria、V. Panneershelvam、M. Suleyman、C. Beattie、S. Petersen、S. Legg、V. Mnih、K. Kavukcuoglu 和 D. 银。深度强化学习的大规模并行方法。在 *Deep Learning Workshop, ICML*, 2015 年。M. Riedmiller。神经拟合 Q 迭代 - 数据高效神经强化学习方法的首次体验。 J. Gama、R. Camacho、P. Brazdil、A. Jorge 和 L. Torgo, 编辑, *Proceedings of the 16th European Conference on Machine Learning (ECML'05)*, 第 317-328 页。Springer, 2005。B. Sallans 和 G. E. Hinton。具有分解状态和动作的强化学习。 *The Journal of Machine Learning Research*, 5: 1063–1088, 2004 年。A. L. Strehl、L. Li 和 M. L. Littman。有限 MDP 中的强化学习：PAC 分析。 *The Journal of Machine Learning Research*, 10: 2413–2444, 2009。R.S.萨顿。学习通过时间差异的方法进行预测。 *Machine learning*, 3(1): 9–44, 1988。R.S.萨顿。基于近似动态规划的学习、规划和反应集成架构。 *Proceedings of the seventh international conference on machine learning*, 第 216-224 页, 1990 年。R. S. Sutton 和 A. G. Barto。 *Introduction to reinforcement learning*。麻省理工学院出版社, 1998 年。R. S. Sutton、C. Szepesvári 和 H. R. Maei。一种采用线性函数逼近的离策略时间差分学习的收敛 $O(n)$ 算法。 *Advances in Neural Information Processing Systems 21 (NIPS-08)*, 21: 1609–1616, 2008。R.S.萨顿、A.R.马哈茂德和M.怀特。解决非策略时间差异学习问题的一种强调方法。 *arXiv preprint arXiv:1503.04269*, 2015。I. Szita 和 A. Lőrincz。乐观主义的多面性：统一的方法。在 *Proceedings of the 25th international conference on Machine learning*, 第 1048-1055 页。ACM, 2008。G. Tesauro。时间差异学习和 td-gammon。 *Communications of the ACM*, 38(3):58–68, 1995。S. Thrun 和 A. Schwartz。使用函数逼近进行强化学习的问题。载于 M. Mozer、P. Smolensky、D. Touretzky、J. Elman 和 A. Weigend, 编辑, *Proceedings of the 1993 Connectionist Models Summer School*, 新泽西州希尔斯代尔, 1993 年。Lawrence Erlbaum。J. N. Tsitsiklis 和 B. Van Roy。函数逼近的时差学习分析。 *IEEE Transactions on Automatic Control*, 42(5): 674–690, 1997。H.范哈塞尔特。双Q学习。 *Advances in Neural Information Processing Systems*, 23: 2613–2621, 2010。H.范哈塞尔特。 *Insights in Reinforcement Learning*。博士论文，乌得勒支大学，2011 年。C. J. C. H. Watkins。 *Learning from delayed rewards*。博士论文，英国剑桥大学，1989。

Appendix

Theorem 1. Consider a state s in which all the true optimal action values are equal at $Q_*(s, a) = V_*(s)$ for some $V_*(s)$. Let Q_t be arbitrary value estimates that are on the whole unbiased in the sense that $\sum_a (Q_t(s, a) - V_*(s)) = 0$, but that are not all zero, such that $\frac{1}{m} \sum_a (Q_t(s, a) - V_*(s))^2 = C$ for some $C > 0$, where $m \geq 2$ is the number of actions in s . Under these conditions, $\max_a Q_t(s, a) \geq V_*(s) + \sqrt{\frac{C}{m-1}}$. This lower bound is tight. Under the same conditions, the lower bound on the absolute error of the Double Q-learning estimate is zero.

Proof of Theorem 1. Define the errors for each action a as $\epsilon_a = Q_t(s, a) - V_*(s)$. Suppose that there exists a setting of $\{\epsilon_a\}$ such that $\max_a \epsilon_a < \sqrt{\frac{C}{m-1}}$. Let $\{\epsilon_i^+\}$ be the set of positive ϵ of size n , and $\{\epsilon_j^-\}$ the set of strictly negative ϵ of size $m - n$ (such that $\{\epsilon\} = \{\epsilon_i^+\} \cup \{\epsilon_j^-\}$). If $n = m$, then $\sum_a \epsilon_a = 0 \implies \epsilon_a = 0 \forall a$, which contradicts $\sum_a \epsilon_a^2 = mC$. Hence, it must be that $n \leq m - 1$. Then, $\sum_{i=1}^n \epsilon_i^+ \leq n \max_i \epsilon_i^+ < n \sqrt{\frac{C}{m-1}}$, and therefore (using the constraint $\sum_a \epsilon_a = 0$) we also have that $\sum_{j=1}^{m-n} |\epsilon_j^-| < n \sqrt{\frac{C}{m-1}}$. This implies $\max_j |\epsilon_j^-| < n \sqrt{\frac{C}{m-1}}$. By Hölder's inequality, then

$$\begin{aligned} \sum_{j=1}^{m-n} (\epsilon_j^-)^2 &\leq \sum_{j=1}^{m-n} |\epsilon_j^-| \cdot \max_j |\epsilon_j^-| \\ &< n \sqrt{\frac{C}{m-1}} n \sqrt{\frac{C}{m-1}}. \end{aligned}$$

We can now combine these relations to compute an upper-bound on the sum of squares for all ϵ_a :

$$\begin{aligned} \sum_{a=1}^m (\epsilon_a)^2 &= \sum_{i=1}^n (\epsilon_i^+)^2 + \sum_{j=1}^{m-n} (\epsilon_j^-)^2 \\ &< n \frac{C}{m-1} + n \sqrt{\frac{C}{m-1}} n \sqrt{\frac{C}{m-1}} \\ &= C \frac{n(n+1)}{m-1} \\ &\leq mC. \end{aligned}$$

This contradicts the assumption that $\sum_{a=1}^m \epsilon_a^2 < mC$, and therefore $\max_a \epsilon_a \geq \sqrt{\frac{C}{m-1}}$ for all settings of ϵ that satisfy the constraints. We can check that the lower-bound is tight by setting $\epsilon_a = \sqrt{\frac{C}{m-1}}$ for $a = 1, \dots, m-1$ and $\epsilon_m = -\sqrt{(m-1)C}$.

This verifies $\sum_a \epsilon_a^2 = mC$ and $\sum_a \epsilon_a = 0$.

The only tight lower bound on the absolute error for Double Q-learning $|Q'_t(s, \arg\max_a Q_t(s, a)) - V_*(s)|$ is zero. This can be seen by because we can have

$$Q_t(s, a_1) = V_*(s) + \sqrt{C \frac{m-1}{m}},$$

and

$$Q_t(s, a_i) = V_*(s) - \sqrt{C \frac{1}{m(m-1)}}, \text{ for } i > 1.$$

Then the conditions of the theorem hold. If then, furthermore, we have $Q'_t(s, a_1) = V_*(s)$ then the error is zero. The remaining action values $Q'_t(s, a_i)$, for $i > 1$, are arbitrary. $\square$

Theorem 2. Consider a state s in which all the true optimal action values are equal at $Q_*(s, a) = V_*(s)$. Suppose that the estimation errors $Q_t(s, a) - Q_*(s, a)$ are independently distributed uniformly randomly in $[-1, 1]$. Then,

$$\mathbb{E} \left[\max_a Q_t(s, a) - V_*(s) \right] = \frac{m-1}{m+1}$$

Proof. Define $\epsilon_a = Q_t(s, a) - Q_*(s, a)$; this is a uniform random variable in $[-1, 1]$. The probability that $\max_a Q_t(s, a) \leq x$ for some x is equal to the probability that $\epsilon_a \leq x$ for all a simultaneously. Because the estimation errors are independent, we can derive

$$\begin{aligned} P(\max_a \epsilon_a \leq x) &= P(X_1 \leq x \wedge X_2 \leq x \wedge \dots \wedge X_m \leq x) \\ &= \prod_{a=1}^m P(\epsilon_a \leq x). \end{aligned}$$

The function $P(\epsilon_a \leq x)$ is the cumulative distribution function (CDF) of ϵ_a , which here is simply defined as

$$P(\epsilon_a \leq x) = \begin{cases} 0 & \text{if } x \leq -1 \\ \frac{1+x}{2} & \text{if } x \in (-1, 1) \\ 1 & \text{if } x \geq 1 \end{cases}$$

This implies that

$$\begin{aligned} P(\max_a \epsilon_a \leq x) &= \prod_{a=1}^m P(\epsilon_a \leq x) \\ &= \begin{cases} 0 & \text{if } x \leq -1 \\ \left(\frac{1+x}{2}\right)^m & \text{if } x \in (-1, 1) \\ 1 & \text{if } x \geq 1 \end{cases} \end{aligned}$$

This gives us the CDF of the random variable $\max_a \epsilon_a$. Its expectation can be written as an integral

$$\mathbb{E} \left[\max_a \epsilon_a \right] = \int_{-1}^1 x f_{\max}(x) dx,$$

where $f_{\max}$ is the probability density function of this variable, defined as the derivative of the CDF: $f_{\max}(x) = \frac{d}{dx} P(\max_a \epsilon_a \leq x)$, so that for $x \in [-1, 1]$ we have $f_{\max}(x) = \frac{m}{2} \left(\frac{1+x}{2}\right)^{m-1}$. Evaluating the integral yields

$$\begin{aligned} \mathbb{E} \left[\max_a \epsilon_a \right] &= \int_{-1}^1 x f_{\max}(x) dx \\ &= \left[\left(\frac{x+1}{2}\right)^m \frac{mx-1}{m+1} \right]_{-1}^1 \\ &= \frac{m-1}{m+1}. \end{aligned} \quad \square$$

Experimental Details for the Atari 2600 Domain

We selected the 49 games to match the list used by Mnih et al. (2015), see Tables below for the full list. Each agent step is composed of four frames (the last selected action is repeated during these frames) and reward values (obtained from the Arcade Learning Environment (Bellemare et al., 2013)) are clipped between -1 and 1.

附录

定理1. Consider a state s in which all the true optimal action values are equal at $Q_*(s, a) = V_*(s)$ for some $V_*(s)$. Let Q_t be arbitrary value estimates that are on the whole unbiased in the sense that $\sum_a (Q_t(s, a) - V_*(s)) = 0$, but that are not all zero, such that $\frac{1}{m} \sum_a (Q_t(s, a) - V_*(s))^2 = C$ for some $C > 0$, where $m \geq 2$ is the number of actions in s . Under these conditions, $\max_a Q_t(s, a) \geq V_*(s) + \sqrt{\frac{C}{m-1}}$. This lower bound is tight. Under the same conditions, the lower bound on the absolute error of the Double Q-learning estimate is zero.

Proof of Theorem 1. 将每个操作 a 的错误定义为 $\epsilon_a = Q_t(s, a) - V_*(s)$ 。假设存在 $\{\epsilon_a\}$ 的设置，使得 $\max_a \epsilon_a < \sqrt{\frac{C}{m-1}}$ 。令 $\{\epsilon_i^+\}$ 为大小为 n 的正数 ϵ 的集合，而 $\{\epsilon_j^-\}$ 为大小为 $m - n$ 的严格负数 ϵ 的集合，使得 $\{\epsilon\} = \{\epsilon_i^+\} \cup \{\epsilon_j^-\}$ 。如果 $n = m$ ，则 $\sum_a \epsilon_a = 0 \Rightarrow \epsilon_a = 0 \forall a$ ，这与 $\sum_a \epsilon_a^2 = mC$ 矛盾。因此，它必须是 $n \leq m - 1$ 。然后， $\sum_{i=1}^n \epsilon_i^+ \leq n \max_i \epsilon_i^+ < n \sqrt{\frac{C}{m-1}}$ 。因此（使用约束 $\sum_a \epsilon_a = 0$ ）我们也有 $\sum_{j=1}^{m-n} |\epsilon_j^-| < n \sqrt{\frac{C}{m-1}}$ 。这意味着 $\max_j |\epsilon_j^-| < n \sqrt{\frac{C}{m-1}}$ 。根据霍尔德不等式，那么

$$\begin{aligned} \sum_{j=1}^{m-n} (\epsilon_j^-)^2 &\leq \sum_{j=1}^{m-n} |\epsilon_j^-| \cdot \max_j |\epsilon_j^-| \\ &< n \sqrt{\frac{C}{m-1}} n \sqrt{\frac{C}{m-1}}. \end{aligned}$$

我们现在可以结合这些关系来计算所有 ϵ_a 的平方和的上限：

$$\begin{aligned} \sum_{a=1}^m (\epsilon_a)^2 &= \sum_{i=1}^n (\epsilon_i^+)^2 + \sum_{j=1}^{m-n} (\epsilon_j^-)^2 \\ &< n \frac{C}{m-1} + n \sqrt{\frac{C}{m-1}} n \sqrt{\frac{C}{m-1}} \\ &= C \frac{n(n+1)}{m-1} \\ &\leq mC. \end{aligned}$$

这与 $\sum_{a=1}^m \epsilon_a^2 < mC$ 的假设相矛盾，因此与满足约束的 ϵ 的所有设置的 $\max_a \epsilon_a \geq \sqrt{\frac{C}{m-1}}$ 相矛盾。我们可以通过为 $a = 1, \dots, m-1$ 和 $\epsilon_m = -\sqrt{(m-1)C}$ 设置 $\epsilon_a = \sqrt{\frac{C}{m-1}}$ 来检查下界是否严格。这验证了 $\sum$

$$\epsilon_a^2 = mC \text{ 和 } \sum_a \epsilon_a = 0.$$

Double Q-learning $|Q'_t(s, \arg\max_a Q_t(s, a)) - V_*(s)|$ 的绝对误差的唯一严格下界为零。这可以看出，因为我们可以有

$$Q_t(s, a_1) = V_*(s) + \sqrt{C \frac{m-1}{m}},$$

和

$$Q_t(s, a_i) = V_*(s) - \sqrt{C \frac{1}{m(m-1)}}, \text{ for } i > 1.$$

那么定理的条件成立。此外，如果我们有 $Q'_t(s, a_1) = V_*(s)$ ，那么误差为零。对于 $i > 1$ ，剩余的动作值 $Q'_t(s, a_i)$ 是任意的。□

定理2. Consider a state s in which all the true optimal action values are equal at $Q_*(s, a) = V_*(s)$. Suppose that the estimation errors $Q_t(s, a) - Q_*(s, a)$ are independently distributed uniformly randomly in $[-1, 1]$. Then,

$$\mathbb{E} \left[\max_a Q_t(s, a) - V_*(s) \right] = \frac{m-1}{m+1}$$

Proof. 定义 $\epsilon_a = Q_t(s, a) - Q_*(s, a)$ ；这是 $[-1, 1]$ 中的统一随机变量。某些 x 的 $\max_a Q_t(s, a) \leq x$ 的概率等于同时所有 a 的 $\epsilon_a \leq x$ 的概率。由于估计误差是独立的，我们可以得出

$$\begin{aligned} P(\max_a \epsilon_a \leq x) &= P(X_1 \leq x \wedge X_2 \leq x \wedge \dots \wedge X_m \leq x) \\ &= \prod_{a=1}^m P(\epsilon_a \leq x). \end{aligned}$$

函数 $P(\epsilon_a \leq x)$ 是 ϵ_a 的累积分布函数（CDF），这里简单地定义为

$$P(\epsilon_a \leq x) = \begin{cases} 0 & \text{if } x \leq -1 \\ \frac{1+x}{2} & \text{if } x \in (-1, 1) \\ 1 & \text{if } x \geq 1 \end{cases}$$

这意味着

$$\begin{aligned} P(\max_a \epsilon_a \leq x) &= \prod_{a=1}^m P(\epsilon_a \leq x) \\ &= \begin{cases} 0 & \text{if } x \leq -1 \\ \left(\frac{1+x}{2}\right)^m & \text{if } x \in (-1, 1) \\ 1 & \text{if } x \geq 1 \end{cases} \end{aligned}$$

这给了我们随机变量 $\max_a \epsilon_a$ 的 CDF。它的期望可以写成一个积分

$$\mathbb{E} \left[\max_a \epsilon_a \right] = \int_{-1}^1 x f_{\max}(x) dx,$$

其中 $f_{\max}$ 是该变量的概率密度函数，定义为 CDF 的导数： $f_{\max}(x) = \frac{d}{dx} P(\max_a \epsilon_a \leq x)$ ，因此对于 $x \in [-1, 1]$ ，我们有 $f_{\max}(x) = \frac{m}{2} \left(\frac{1+x}{2}\right)^{m-1}$ 。评估积分收益率 — —

$$\begin{aligned} \mathbb{E} \left[\max_a \epsilon_a \right] &= \int_{-1}^1 x f_{\max}(x) dx \\ &= \left[\left(\frac{x+1}{2}\right)^m \frac{mx-1}{m+1} \right]_{-1}^1 \\ &= \frac{m-1}{m+1}. \end{aligned} \quad \square$$

Atari 2600 域的实验细节

我们选择了 49 款游戏来匹配 Mnih 等人使用的列表。(2015)，完整列表请参见下表。每个代理步骤由四个帧组成（在这些帧期间重复最后选择的动作），并且奖励值（从 Arcade 学习环境（Bellemare 等人，2013）获得）被限制在 -1 和 1 之间。

Network Architecture

The convolution network used in the experiment is exactly the one proposed by Mnih et al. (2015), we only provide details here for completeness. The input to the network is a $84 \times 84 \times 4$ tensor containing a rescaled, and gray-scale, version of the last four frames. The first convolution layer convolves the input with 32 filters of size 8 (stride 4), the second layer has 64 layers of size 4 (stride 2), the final convolution layer has 64 filters of size 3 (stride 1). This is followed by a fully-connected hidden layer of 512 units. All these layers are separated by Rectifier Linear Units (ReLU). Finally, a fully-connected linear layer projects to the output of the network, i.e., the Q-values. The optimization employed to train the network is RMSProp (with momentum parameter 0.95).

Hyper-parameters

In all experiments, the discount was set to $\gamma = 0.99$, and the learning rate to $\alpha = 0.00025$. The number of steps between target network updates was $\tau = 10,000$. Training is done over 50M steps (i.e., 200M frames). The agent is evaluated every 1M steps, and the best policy across these evaluations is kept as the output of the learning process. The size of the experience replay memory is 1M tuples. The memory gets sampled to update the network every 4 steps with minibatches of size 32. The simple exploration policy used is an ϵ -greedy policy with the ϵ decreasing linearly from 1 to 0.1 over 1M steps.

Supplementary Results in the Atari 2600 Domain

The Tables below provide further detailed results for our experiments in the Atari domain.

网络架构

实验中使用的卷积网络正是Mnih等人提出的卷积网络。（2015），为了完整性，我们在这里仅提供细节。网络的输入是一个 $84 \times 84 \times 4$ 张量，包含最后四帧的重新缩放和灰度版本。第一个卷积层将输入与 32 个大小为 8（步长 4）的滤波器进行卷积，第二层有 64 个大小为 4（步长 2）的层，最后一个卷积层有 64 个大小为 3（步长 1）的滤波器。接下来是 512 个单元的全连接隐藏层。所有这些层均由整流线性单元 (ReLU) 分隔。最后，全连接线性层投影到网络的输出，即 Q 值。用于训练网络的优化是 RMSProp（动量参数为 0.95）。

超参数

在所有实验中，折扣设置为 $\gamma = 0.99$ ，学习率设置为 $\alpha = 0.00025$ 。目标网络更新之间的步数为 $\tau = 10,000$ 。训练超过 50 M 步骤（即 200M 帧）。每 1M 步对代理进行一次评估，并且这些评估中的最佳策略将保留为学习过程的输出。经验回放内存的大小是 1M 元组。对内存进行采样，以每 4 个步骤更新网络，并使用大小为 32 的小批量。使用的简单探索策略是 ϵ 贪婪策略，其中 ϵ 在 1M 步内从 1 线性减小到 0.1。

Atari 2600 领域的补充结果

下表提供了我们在 Atari 领域进行的实验的更详细的结果。

Game	Random	Human	DQN	Double DQN
Alien	227.80	6875.40	3069.33	2907.30
Amidar	5.80	1675.80	739.50	702.10
Assault	222.40	1496.40	3358.63	5022.90
Asterix	210.00	8503.30	6011.67	15150.00
Asteroids	719.10	13156.70	1629.33	930.60
Atlantis	12850.00	29028.10	85950.00	64758.00
Bank Heist	14.20	734.40	429.67	728.30
Battle Zone	2360.00	37800.00	26300.00	25730.00
Beam Rider	363.90	5774.70	6845.93	7654.00
Bowling	23.10	154.80	42.40	70.50
Boxing	0.10	4.30	71.83	81.70
Breakout	1.70	31.80	401.20	375.00
Centipede	2090.90	11963.20	8309.40	4139.40
Chopper Command	811.00	9881.80	6686.67	4653.00
Crazy Climber	10780.50	35410.50	114103.33	101874.00
Demon Attack	152.10	3401.30	9711.17	9711.90
Double Dunk	-18.60	-15.50	-18.07	-6.30
Enduro	0.00	309.60	301.77	319.50
Fishing Derby	-91.70	5.50	-0.80	20.30
Freeway	0.00	29.60	30.30	31.80
Frostbite	65.20	4334.70	328.33	241.50
Gopher	257.60	2321.00	8520.00	8215.40
Gravitar	173.00	2672.00	306.67	170.50
H.E.R.O.	1027.00	25762.50	19950.33	20357.00
Ice Hockey	-11.20	0.90	-1.60	-2.40
James Bond	29.00	406.70	576.67	438.00
Kangaroo	52.00	3035.00	6740.00	13651.00
Krull	1598.00	2394.60	3804.67	4396.70
Kung-Fu Master	258.50	22736.20	23270.00	29486.00
Montezuma's Revenge	0.00	4366.70	0.00	0.00
Ms. Pacman	307.30	15693.40	2311.00	3210.00
Name This Game	2292.30	4076.20	7256.67	6997.10
Pong	-20.70	9.30	18.90	21.00
Private Eye	24.90	69571.30	1787.57	670.10
Q*Bert	163.90	13455.00	10595.83	14875.00
River Raid	1338.50	13513.30	8315.67	12015.30
Road Runner	11.50	7845.00	18256.67	48377.00
Robotank	2.20	11.90	51.57	46.70
Seaquest	68.40	20181.80	5286.00	7995.00
Space Invaders	148.00	1652.30	1975.50	3154.60
Star Gunner	664.00	10250.00	57996.67	65188.00
Tennis	-23.80	-8.90	-2.47	1.70
Time Pilot	3568.00	5925.00	5946.67	7964.00
Tutankham	11.40	167.60	186.70	190.60
Up and Down	533.40	9082.00	8456.33	16769.90
Venture	0.00	1187.50	380.00	93.00
Video Pinball	16256.90	17297.60	42684.07	70009.00
Wizard of Wor	563.50	4756.50	3393.33	5204.00
Zaxxon	32.50	9173.30	4976.67	10182.00

Table 3: Raw scores for the no-op evaluation condition (5 minutes emulator time). DQN as given by Mnih et al. (2015).

Game	Random	Human	DQN	Double DQN
Alien	227.80	6875.40	3069.33	2907.30
Amidar	5.80	1675.80	739.50	702.10
Assault	222.40	1496.40	3358.63	5022.90
Asterix	210.00	8503.30	6011.67	15150.00
Asteroids	719.10	13156.70	1629.33	930.60
Atlantis	12850.00	29028.10	85950.00	64758.00
Bank Heist	14.20	734.40	429.67	728.30
Battle Zone	2360.00	37800.00	26300.00	25730.00
Beam Rider	363.90	5774.70	6845.93	7654.00
Bowling	23.10	154.80	42.40	70.50
Boxing	0.10	4.30	71.83	81.70
Breakout	1.70	31.80	401.20	375.00
Centipede	2090.90	11963.20	8309.40	4139.40
Chopper Command	811.00	9881.80	6686.67	4653.00
Crazy Climber	10780.50	35410.50	114103.33	101874.00
Demon Attack	152.10	3401.30	9711.17	9711.90
Double Dunk	-18.60	-15.50	-18.07	-6.30
Enduro	0.00	309.60	301.77	319.50
Fishing Derby	-91.70	5.50	-0.80	20.30
Freeway	0.00	29.60	30.30	31.80
Frostbite	65.20	4334.70	328.33	241.50
Gopher	257.60	2321.00	8520.00	8215.40
Gravitar	173.00	2672.00	306.67	170.50
H.E.R.O.	1027.00	25762.50	19950.33	20357.00
Ice Hockey	-11.20	0.90	-1.60	-2.40
James Bond	29.00	406.70	576.67	438.00
Kangaroo	52.00	3035.00	6740.00	13651.00
Krull	1598.00	2394.60	3804.67	4396.70
Kung-Fu Master	258.50	22736.20	23270.00	29486.00
Montezuma's Revenge	0.00	4366.70	0.00	0.00
Ms. Pacman	307.30	15693.40	2311.00	3210.00
Name This Game	2292.30	4076.20	7256.67	6997.10
Pong	-20.70	9.30	18.90	21.00
Private Eye	24.90	69571.30	1787.57	670.10
Q*Bert	163.90	13455.00	10595.83	14875.00
River Raid	1338.50	13513.30	8315.67	12015.30
Road Runner	11.50	7845.00	18256.67	48377.00
Robotank	2.20	11.90	51.57	46.70
Seaquest	68.40	20181.80	5286.00	7995.00
Space Invaders	148.00	1652.30	1975.50	3154.60
Star Gunner	664.00	10250.00	57996.67	65188.00
Tennis	-23.80	-8.90	-2.47	1.70
Time Pilot	3568.00	5925.00	5946.67	7964.00
Tutankham	11.40	167.60	186.70	190.60
Up and Down	533.40	9082.00	8456.33	16769.90
Venture	0.00	1187.50	380.00	93.00
Video Pinball	16256.90	17297.60	42684.07	70009.00
Wizard of Wor	563.50	4756.50	3393.33	5204.00
Zaxxon	32.50	9173.30	4976.67	10182.00

表 3: 无操作评估条件的原始分数 (5 分钟仿真器时间)。Mnih 等人给出的 DQN。(2015)。

Game	DQN	Double DQN
Alien	42.75 %	40.31 %
Amidar	43.93 %	41.69 %
Assault	246.17 %	376.81 %
Asterix	69.96 %	180.15 %
Asteroids	7.32 %	1.70 %
Atlantis	451.85 %	320.85 %
Bank Heist	57.69 %	99.15 %
Battle Zone	67.55 %	65.94 %
Beam Rider	119.80 %	134.73 %
Bowling	14.65 %	35.99 %
Boxing	1707.86 %	1942.86 %
Breakout	1327.24 %	1240.20 %
Centipede	62.99 %	20.75 %
Chopper Command	64.78 %	42.36 %
Crazy Climber	419.50 %	369.85 %
Demon Attack	294.20 %	294.22 %
Double Dunk	17.10 %	396.77 %
Enduro	97.47 %	103.20 %
Fishing Derby	93.52 %	115.23 %
Freeway	102.36 %	107.43 %
Frostbite	6.16 %	4.13 %
Gopher	400.43 %	385.66 %
Gravitar	5.35 %	-0.10 %
H.E.R.O.	76.50 %	78.15 %
Ice Hockey	79.34 %	72.73 %
James Bond	145.00 %	108.29 %
Kangaroo	224.20 %	455.88 %
Krull	277.01 %	351.33 %
Kung-Fu Master	102.37 %	130.03 %
Montezuma's Revenge	0.00 %	0.00 %
Ms. Pacman	13.02 %	18.87 %
Name This Game	278.29 %	263.74 %
Pong	132.00 %	139.00 %
Private Eye	2.53 %	0.93 %
Q*Bert	78.49 %	110.68 %
River Raid	57.31 %	87.70 %
Road Runner	232.91 %	617.42 %
Robotank	508.97 %	458.76 %
Seaquest	25.94 %	39.41 %
Space Invaders	121.49 %	199.87 %
Star Gunner	598.09 %	673.11 %
Tennis	143.15 %	171.14 %
Time Pilot	100.92 %	186.51 %
Tutankham	112.23 %	114.72 %
Up and Down	92.68 %	189.93 %
Venture	32.00 %	7.83 %
Video Pinball	2539.36 %	5164.99 %
Wizard of Wor	67.49 %	110.67 %
Zaxxon	54.09 %	111.04 %

Table 4: Normalized results for no-op evaluation condition (5 minutes emulator time).

Game	DQN	Double DQN
Alien	42.75 %	40.31 %
Amidar	43.93 %	41.69 %
Assault	246.17 %	376.81 %
Asterix	69.96 %	180.15 %
Asteroids	7.32 %	1.70 %
Atlantis	451.85 %	320.85 %
Bank Heist	57.69 %	99.15 %
Battle Zone	67.55 %	65.94 %
Beam Rider	119.80 %	134.73 %
Bowling	14.65 %	35.99 %
Boxing	1707.86 %	1942.86 %
Breakout	1327.24 %	1240.20 %
Centipede	62.99 %	20.75 %
Chopper Command	64.78 %	42.36 %
Crazy Climber	419.50 %	369.85 %
Demon Attack	294.20 %	294.22 %
Double Dunk	17.10 %	396.77 %
Enduro	97.47 %	103.20 %
Fishing Derby	93.52 %	115.23 %
Freeway	102.36 %	107.43 %
Frostbite	6.16 %	4.13 %
Gopher	400.43 %	385.66 %
Gravitar	5.35 %	-0.10 %
H.E.R.O.	76.50 %	78.15 %
Ice Hockey	79.34 %	72.73 %
James Bond	145.00 %	108.29 %
Kangaroo	224.20 %	455.88 %
Krull	277.01 %	351.33 %
Kung-Fu Master	102.37 %	130.03 %
Montezuma's Revenge	0.00 %	0.00 %
Ms. Pacman	13.02 %	18.87 %
Name This Game	278.29 %	263.74 %
Pong	132.00 %	139.00 %
Private Eye	2.53 %	0.93 %
Q*Bert	78.49 %	110.68 %
River Raid	57.31 %	87.70 %
Road Runner	232.91 %	617.42 %
Robotank	508.97 %	458.76 %
Seaquest	25.94 %	39.41 %
Space Invaders	121.49 %	199.87 %
Star Gunner	598.09 %	673.11 %
Tennis	143.15 %	171.14 %
Time Pilot	100.92 %	186.51 %
Tutankham	112.23 %	114.72 %
Up and Down	92.68 %	189.93 %
Venture	32.00 %	7.83 %
Video Pinball	2539.36 %	5164.99 %
Wizard of Wor	67.49 %	110.67 %
Zaxxon	54.09 %	111.04 %

表 4: 无操作评估条件的归一化结果 (5 分钟仿真器时间)。

Game	Random	Human	DQN	Double DQN	Double DQN (tuned)
Alien	128.30	6371.30	570.2	621.6	1033.4
Amidar	11.80	1540.40	133.4	188.2	169.1
Assault	166.90	628.90	3332.3	2774.3	6060.8
Asterix	164.50	7536.00	124.5	5285.0	16837.0
Asteroids	871.30	36517.30	697.1	1219.0	1193.2
Atlantis	13463.00	26575.00	76108.0	260556.0	319688.0
Bank Heist	21.70	644.50	176.3	469.8	886.0
Battle Zone	3560.00	33030.00	17560.0	25240.0	24740.0
Beam Rider	254.60	14961.00	8672.4	9107.9	17417.2
Berzerk	196.10	2237.50		635.8	1011.1
Bowling	35.20	146.50	41.2	62.3	69.6
Boxing	-1.50	9.60	25.8	52.1	73.5
Breakout	1.60	27.90	303.9	338.7	368.9
Centipede	1925.50	10321.90	3773.1	5166.6	3853.5
Chopper Command	644.00	8930.00	3046.0	2483.0	3495.0
Crazy Climber	9337.00	32667.00	50992.0	94315.0	113782.0
Defender	1965.50	14296.00		8531.0	27510.0
Demon Attack	208.30	3442.80	12835.2	13943.5	69803.4
Double Dunk	-16.00	-14.40	-21.6	-6.4	-0.3
Enduro	-81.80	740.20	475.6	475.9	1216.6
Fishing Derby	-77.10	5.10	-2.3	-3.4	3.2
Freeway	0.10	25.60	25.8	26.3	28.8
Frostbite	66.40	4202.80	157.4	258.3	1448.1
Gopher	250.00	2311.00	2731.8	8742.8	15253.0
Gravitar	245.50	3116.00	216.5	170.0	200.5
H.E.R.O.	1580.30	25839.40	12952.5	15341.4	14892.5
Ice Hockey	-9.70	0.50	-3.8	-3.6	-2.5
James Bond	33.50	368.50	348.5	416.0	573.0
Kangaroo	100.00	2739.00	2696.0	6138.0	11204.0
Krull	1151.90	2109.10	3864.0	6130.4	6796.1
Kung-Fu Master	304.00	20786.80	11875.0	22771.0	30207.0
Montezuma's Revenge	25.00	4182.00	50.0	30.0	42.0
Ms. Pacman	197.80	15375.00	763.5	1401.8	1241.3
Name This Game	1747.80	6796.00	5439.9	7871.5	8960.3
Phoenix	1134.40	6686.20		10364.0	12366.5
Pit Fall	-348.80	5998.90		-432.9	-186.7
Pong	-18.00	15.50	16.2	17.7	19.1
Private Eye	662.80	64169.10	298.2	346.3	-575.5
Q*Bert	183.00	12085.00	4589.8	10713.3	11020.8
River Raid	588.30	14382.20	4065.3	6579.0	10838.4
Road Runner	200.00	6878.00	9264.0	43884.0	43156.0
Robotank	2.40	8.90	58.5	52.0	59.1
Seaquest	215.50	40425.80	2793.9	4199.4	14498.0
Skiing	-15287.40	-3686.60		-29404.3	-11490.4
Solaris	2047.20	11032.60		2166.8	810.0
Space Invaders	182.60	1464.90	1449.7	1495.7	2628.7
Star Gunner	697.00	9528.00	34081.0	53052.0	58365.0
Surround	-9.70	5.40		-7.6	1.9
Tennis	-21.40	-6.70	-2.3	11.0	-7.8
Time Pilot	3273.00	5650.00	5640.0	5375.0	6608.0
Tutankham	12.70	138.30	32.4	63.6	92.2
Up and Down	707.20	9896.10	3311.3	4721.1	19086.9
Venture	18.00	1039.00	54.0	75.0	21.0
Video Pinball	20452.0	15641.10	20228.1	148883.6	367823.7
Wizard of Wor	804.00	4556.00	246.0	155.0	6201.0
Yars Revenge	1476.90	47135.20		5439.5	6270.6
Zaxxon	475.00	8443.00	831.0	7874.0	8593.0

Table 5: Raw scores for the human start condition (30 minutes emulator time). DQN as given by Nair et al. (2015).

Game	Random	Human	DQN	Double DQN	Double DQN (tuned)
Alien	128.30	6371.30	570.2	621.6	1033.4
Amidar	11.80	1540.40	133.4	188.2	169.1
Assault	166.90	628.90	3332.3	2774.3	6060.8
Asterix	164.50	7536.00	124.5	5285.0	16837.0
Asteroids	871.30	36517.30	697.1	1219.0	1193.2
Atlantis	13463.00	26575.00	76108.0	260556.0	319688.0
Bank Heist	21.70	644.50	176.3	469.8	886.0
Battle Zone	3560.00	33030.00	17560.0	25240.0	24740.0
Beam Rider	254.60	14961.00	8672.4	9107.9	17417.2
Berzerk	196.10	2237.50		635.8	1011.1
Bowling	35.20	146.50	41.2	62.3	69.6
Boxing	-1.50	9.60	25.8	52.1	73.5
Breakout	1.60	27.90	303.9	338.7	368.9
Centipede	1925.50	10321.90	3773.1	5166.6	3853.5
Chopper Command	644.00	8930.00	3046.0	2483.0	3495.0
Crazy Climber	9337.00	32667.00	50992.0	94315.0	113782.0
Defender	1965.50	14296.00		8531.0	27510.0
Demon Attack	208.30	3442.80	12835.2	13943.5	69803.4
Double Dunk	-16.00	-14.40	-21.6	-6.4	-0.3
Enduro	-81.80	740.20	475.6	475.9	1216.6
Fishing Derby	-77.10	5.10	-2.3	-3.4	3.2
Freeway	0.10	25.60	25.8	26.3	28.8
Frostbite	66.40	4202.80	157.4	258.3	1448.1
Gopher	250.00	2311.00	2731.8	8742.8	15253.0
Gravitar	245.50	3116.00	216.5	170.0	200.5
H.E.R.O.	1580.30	25839.40	12952.5	15341.4	14892.5
Ice Hockey	-9.70	0.50	-3.8	-3.6	-2.5
James Bond	33.50	368.50	348.5	416.0	573.0
Kangaroo	100.00	2739.00	2696.0	6138.0	11204.0
Krull	1151.90	2109.10	3864.0	6130.4	6796.1
Kung-Fu Master	304.00	20786.80	11875.0	22771.0	30207.0
Montezuma's Revenge	25.00	4182.00	50.0	30.0	42.0
Ms. Pacman	197.80	15375.00	763.5	1401.8	1241.3
Name This Game	1747.80	6796.00	5439.9	7871.5	8960.3
Phoenix	1134.40	6686.20		10364.0	12366.5
Pit Fall	-348.80	5998.90		-432.9	-186.7
Pong	-18.00	15.50	16.2	17.7	19.1
Private Eye	662.80	64169.10	298.2	346.3	-575.5
Q*Bert	183.00	12085.00	4589.8	10713.3	11020.8
River Raid	588.30	14382.20	4065.3	6579.0	10838.4
Road Runner	200.00	6878.00	9264.0	43884.0	43156.0
Robotank	2.40	8.90	58.5	52.0	59.1
Seaquest	215.50	40425.80	2793.9	4199.4	14498.0
Skiing	-15287.40	-3686.60		-29404.3	-11490.4
Solaris	2047.20	11032.60		2166.8	810.0
Space Invaders	182.60	1464.90	1449.7	1495.7	2628.7
Star Gunner	697.00	9528.00	34081.0	53052.0	58365.0
Surround	-9.70	5.40		-7.6	1.9
Tennis	-21.40	-6.70	-2.3	11.0	-7.8
Time Pilot	3273.00	5650.00	5640.0	5375.0	6608.0
Tutankham	12.70	138.30	32.4	63.6	92.2
Up and Down	707.20	9896.10	3311.3	4721.1	19086.9
Venture	18.00	1039.00	54.0	75.0	21.0
Video Pinball	20452.0	15641.10	20228.1	148883.6	367823.7
Wizard of Wor	804.00	4556.00	246.0	155.0	6201.0
Yars Revenge	1476.90	47135.20		5439.5	6270.6
Zaxxon	475.00	8443.00	831.0	7874.0	8593.0

表 5: 人类启动条件的原始分数 (30 分钟模拟器时间)。Nair 等人给出的 DQN。(2015)。

Game	DQN	Double DQN	Double DQN (tuned)
Alien	7.08%	7.90%	14.50%
Amidar	7.95%	11.54%	10.29%
Assault	685.15%	564.37%	1275.74%
Asterix	-0.54%	69.46%	226.18%
Asteroids	-0.49%	0.98%	0.90%
Atlantis	477.77%	1884.48%	2335.46%
Bank Heist	24.82%	71.95%	138.78%
Battle Zone	47.51%	73.57%	71.87%
Beam Rider	57.24%	60.20%	116.70%
Berzerk		21.54%	39.92%
Bowling	5.39%	24.35%	30.91%
Boxing	245.95%	482.88%	675.68%
Breakout	1149.43%	1281.75%	1396.58%
Centipede	22.00%	38.60%	22.96%
Chopper Command	28.99%	22.19%	34.41%
Crazy Climber	178.55%	364.24%	447.69%
Defender		53.25%	207.17%
Demon Attack	390.38%	424.65%	2151.65%
Double Dunk	-350.00%	600.00%	981.25%
Enduro	67.81%	67.85%	157.96%
Fishing Derby	91.00%	89.66%	97.69%
Freeway	100.78%	102.75%	112.55%
Frostbite	2.20%	4.64%	33.40%
Gopher	120.42%	412.07%	727.95%
Gravitar	-1.01%	-2.63%	-1.57%
H.E.R.O.	46.88%	56.73%	54.88%
Ice Hockey	57.84%	59.80%	70.59%
James Bond	94.03%	114.18%	161.04%
Kangaroo	98.37%	228.80%	420.77%
Krull	283.34%	520.11%	589.66%
Kung-Fu Master	56.49%	109.69%	145.99%
Montezuma's Revenge	0.60%	0.12%	0.41%
Ms. Pacman	3.73%	7.93%	6.88%
Name This Game	73.14%	121.30%	142.87%
Phoenix		166.25%	202.31%
Pit Fall		-1.32%	2.55%
Pong	102.09%	106.57%	110.75%
Private Eye	-0.57%	-0.50%	-1.95%
Q*Bert	37.03%	88.48%	91.06%
River Raid	25.21%	43.43%	74.31%
Road Runner	135.73%	654.15%	643.25%
Robotank	863.08%	763.08%	872.31%
Seaquest	6.41%	9.91%	35.52%
Skiing		-121.69%	32.73%
Solaris		1.33%	-13.77%
Space Invaders	98.81%	102.40%	190.76%
Star Gunner	378.03%	592.85%	653.02%
Surround		13.91%	76.82%
Tennis	129.93%	220.41%	92.52%
Time Pilot	99.58%	88.43%	140.30%
Tutankham	15.68%	40.53%	63.30%
Up and Down	28.34%	43.68%	200.02%
Venture	3.53%	5.58%	0.29%
Video Pinball	-4.65%	2669.60%	7220.51%
Wizard of Wor	-14.87%	-17.30%	143.84%
Yars Revenge		8.68%	10.50%
Zaxxon	4.47%	92.86%	101.88%

Table 6: Normalized scores for the human start condition (30 minutes emulator time).

Game	DQN	Double DQN	Double DQN (tuned)
Alien	7.08%	7.90%	14.50%
Amidar	7.95%	11.54%	10.29%
Assault	685.15%	564.37%	1275.74%
Asterix	-0.54%	69.46%	226.18%
Asteroids	-0.49%	0.98%	0.90%
Atlantis	477.77%	1884.48%	2335.46%
Bank Heist	24.82%	71.95%	138.78%
Battle Zone	47.51%	73.57%	71.87%
Beam Rider	57.24%	60.20%	116.70%
Berzerk		21.54%	39.92%
Bowling	5.39%	24.35%	30.91%
Boxing	245.95%	482.88%	675.68%
Breakout	1149.43%	1281.75%	1396.58%
Centipede	22.00%	38.60%	22.96%
Chopper Command	28.99%	22.19%	34.41%
Crazy Climber	178.55%	364.24%	447.69%
Defender		53.25%	207.17%
Demon Attack	390.38%	424.65%	2151.65%
Double Dunk	-350.00%	600.00%	981.25%
Enduro	67.81%	67.85%	157.96%
Fishing Derby	91.00%	89.66%	97.69%
Freeway	100.78%	102.75%	112.55%
Frostbite	2.20%	4.64%	33.40%
Gopher	120.42%	412.07%	727.95%
Gravitar	-1.01%	-2.63%	-1.57%
H.E.R.O.	46.88%	56.73%	54.88%
Ice Hockey	57.84%	59.80%	70.59%
James Bond	94.03%	114.18%	161.04%
Kangaroo	98.37%	228.80%	420.77%
Krull	283.34%	520.11%	589.66%
Kung-Fu Master	56.49%	109.69%	145.99%
Montezuma's Revenge	0.60%	0.12%	0.41%
Ms. Pacman	3.73%	7.93%	6.88%
Name This Game	73.14%	121.30%	142.87%
Phoenix		166.25%	202.31%
Pit Fall		-1.32%	2.55%
Pong	102.09%	106.57%	110.75%
Private Eye	-0.57%	-0.50%	-1.95%
Q*Bert	37.03%	88.48%	91.06%
River Raid	25.21%	43.43%	74.31%
Road Runner	135.73%	654.15%	643.25%
Robotank	863.08%	763.08%	872.31%
Seaquest	6.41%	9.91%	35.52%
Skiing		-121.69%	32.73%
Solaris		1.33%	-13.77%
Space Invaders	98.81%	102.40%	190.76%
Star Gunner	378.03%	592.85%	653.02%
Surround		13.91%	76.82%
Tennis	129.93%	220.41%	92.52%
Time Pilot	99.58%	88.43%	140.30%
Tutankham	15.68%	40.53%	63.30%
Up and Down	28.34%	43.68%	200.02%
Venture	3.53%	5.58%	0.29%
Video Pinball	-4.65%	2669.60%	7220.51%
Wizard of Wor	-14.87%	-17.30%	143.84%
Yars Revenge		8.68%	10.50%
Zaxxon	4.47%	92.86%	101.88%

表 6: 人类启动条件的标准化分数 (30 分钟模拟器时间)。